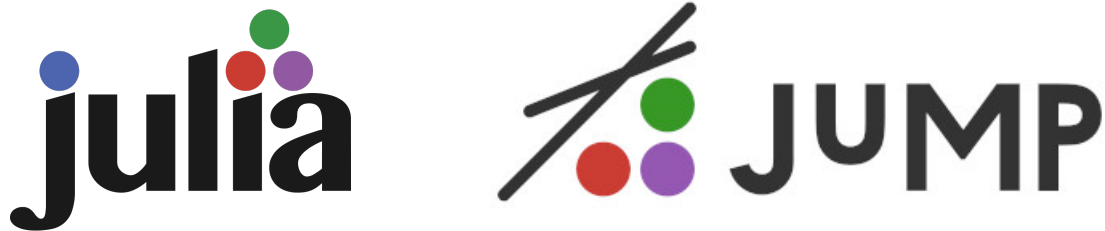


# Mathematical Programming with Julia

An open-source approach to  
Linear & Mixed Integer Programming  
Version 1.3  
Julia 1.8.3 (at least)/JuMP 1.0 (at least)



Stefan Røpke, Evelien Van Der Hurk, Richard  
Lusby & Thomas Stidsen  
VERSION 3  
2026



ISBN: 978-87-93458-25-3

Publisher: DTU Management, Technical University of Denmark, Denmark, 2022

This work is licensed under a Creative Commons  
“Attribution-ShareAlike 4.0 International” license.



*The authors, Richard Lusby & Thomas Stidsen, would like to acknowledge*

*Thomas would like to thank Charlotte*

*Richard would like to thank Angelika*



# 1 Welcome to the course: Course plan

Welcome to the course. This book/pdf-file contains a brief installation chapter, a brief introduction chapter on Linear Programming, a brief chapter on Mixed Integer Programming, and all the exercises of the course. The main body of this book is the Linear programming assignments and Mixed Integer Programming assignments.

In this course you will learn to formulate mathematical optimization models, and write computer programs in Julia to solve them. Lectures will be provided on-site only. Also, support for exercises takes place on-site. Lectures take place in Aud. 13 in building 308. Exercises take place in Building 358, rooms: 069 (main room, with blackboard list for TA's), 070 and 071, where TAs are available for help you with the exercises.

In this course you will learn to create models and implement them into computer programs in Julia, so that you can find their optimal solution. Notice: We will not stream the lectures. If you want to hear any of the lectures, you have to come to Aud. 13 in building 308 at the right time (see below). We will make the slides available the day before the lecture. Every day also consists of a workshop part where you work independently on exercises. TA's are available for help in our main exercise room 069 Building 308. You can also always request an exercise or problem to be reviewed in the next lecture. There will be no help on-line, only on-site.

**As preparation before the course starts install Julia/JuMP and an editor to write programs, e.g. VS Code.**

- For each day of the course, except the last examination day and the catch up day (22/1 2026).
- In the end of each day, we will upload the correct answers to all assignments of the day, 17.00 to the content for that day in DTU-learn: <https://learn.inside.dtu.dk/d21/home/241623>.

This year we are experimenting with some more lecture time. However, the focus of the course is learning by *doing*. Therefore these lectures will take the form of "guided workshops", which means that short explanations will be followed by hands-on exercises that are immediately followed up by feedback on the results. Throughout the course,

you will gradually work more independently and we will reduce the amount of lecture time.

**Notice:** If you feel you need to brush up on Linear Programming, we refer to Chapter 3. In Chapter 5 Mixed Integer Programming is briefly reviewed.

## 1.1 Week 1

### Monday 5/1

1. Lecture 1: Course introduction and Linear Programming (Thomas)
2. Jewellery Production 4.1
3. Micro Brewery 4.2
4. Blending 4.3

### Tuesday 6/1

1. Lecture 2: Basic Julia/JuMP (Stefan)
2. Blending 2 4.4
3. Factory Planning 4.5
4. Project Scheduling 4.6

### Wednesday 7/1

1. Lecture 3: Mixed Integer Programming (Thomas)
2. Young Couple 6.1
3. Startup Fund 6.2
4. Logic 6.3
5. Stamps 6.4, data available on DTU-learn.

## Thursday 8/1

1. Lecture 4: MIP hardness (Stefan)
2. Micro Brewery 2 6.5
3. Santa's Workshop Tour 2019 6.6, data available on DTU-learn.
4. Factory Planning 2 6.7

## Friday 9/1

1. Lecture 5: AI for modelling (Thomas)
2. Student Teacher Meetings 6.8, data available on DTU-learn.
3. Workplan for Teaching Assistants 6.9, data available on DTU-learn.
4. Scrap Removal 6.10

## 1.2 Week 2

### Monday 12/1

1. Lecture 6: Network models (Stefan)
2. Shortest Path [new]
3. Min cost flow [new]
4. Multicommodity flow [new]
5. Network design [new]

### Tuesday 13/1

1. Lecture 7: Vehicle routing (Stefan)
2. Groceries 6.11
3. Tennis 6.12
4. Aircraft Landing 6.13

### Wednesday 14/1

1. Lecture 8: Energy models (Mikkel)
2. Tariff Rates 6.14
3. Railway Depot Shunting 6.15, data available on DTU-Learn

### Thursday 15/1

1. Food Festival 6.16, data available on DTU-learn.
2. The Wedding Planner 6.17, data available on DTU-learn.

### Friday 16/1

1. Hot Air 6.18
2. Wind Farm Cable Routing 6.19, data available on DTU-Learn



## 1.3 Week 3

### Monday 19/1

1. Lecture 9: Math heuristics (Mikkel)
2. Wedding Planner, math-heuristic algorithms 6.20
3. Stamps, math-heuristic algorithms 6.21

### Tuesday 20/1

1. Lecture 10: Multi-Objective Mixed Integer Programming (Thomas)
2. Multi-objective Startup fund 8.1
3. Multi-objective Young Couple 8.2
4. Multi-objective Wedding planning 8.3
5. Multi-objective Aircraft Landing 8.4

### Wednesday 21/1

1. Lecture 11: The exam (Thomas)
2. Exam test (old exam)
3. University Timetabling 6.22, data available on DTU-learn.

### Thursday 22/1

1. Catch up day

### Friday 23/1

Exam day



# Contents

|          |                                                         |           |
|----------|---------------------------------------------------------|-----------|
| <b>1</b> | <b>Welcome to the course: Course plan</b>               | <b>5</b>  |
| 1.1      | Week 1 . . . . .                                        | 6         |
| 1.2      | Week 2 . . . . .                                        | 8         |
| 1.3      | Week 3 . . . . .                                        | 9         |
| <b>2</b> | <b>Installation</b>                                     | <b>15</b> |
| 2.1      | Julia . . . . .                                         | 15        |
| 2.2      | JuMP . . . . .                                          | 15        |
| 2.3      | Solvers . . . . .                                       | 15        |
| 2.4      | Editors . . . . .                                       | 16        |
| 2.5      | Why Julia ? . . . . .                                   | 17        |
| <b>3</b> | <b>Linear Programming</b>                               | <b>19</b> |
| 3.1      | Incredible Chairs . . . . .                             | 19        |
| 3.2      | Incredible Chairs 2 . . . . .                           | 23        |
| 3.3      | Good modelling practices and debugging . . . . .        | 27        |
| 3.3.1    | Debugging . . . . .                                     | 27        |
| 3.4      | Conditional sums and constraints . . . . .              | 28        |
| 3.5      | Balance constraints . . . . .                           | 29        |
| 3.6      | Relaxation and restriction of linear programs . . . . . | 30        |
| 3.7      | Solver parameters . . . . .                             | 30        |
| 3.8      | Julia and JuMP structure . . . . .                      | 31        |
| 3.8.1    | Julia . . . . .                                         | 31        |
| 3.8.2    | JuMP . . . . .                                          | 31        |
| 3.8.3    | Solver . . . . .                                        | 33        |
| 3.8.4    | Structure . . . . .                                     | 33        |
| <b>4</b> | <b>Linear Programming problems</b>                      | <b>37</b> |
| 4.1      | Jewelry Production . . . . .                            | 38        |
| 4.2      | Micro brewery . . . . .                                 | 40        |
| 4.3      | Blending <sup>1</sup> . . . . .                         | 42        |
| 4.4      | Blending 2 <sup>2</sup> . . . . .                       | 44        |

---

<sup>1</sup>Taken from Williams: “Model Building in Mathematical Programming”

<sup>2</sup>Taken from Williams: “Model Building in Mathematical Programming”

|          |                                                       |            |
|----------|-------------------------------------------------------|------------|
| 4.5      | Factory Planning <sup>3</sup>                         | 45         |
| 4.6      | Project Scheduling                                    | 47         |
| <b>5</b> | <b>Mixed Integer Programming</b>                      | <b>51</b>  |
| 5.1      | Modelling power                                       | 52         |
| 5.1.1    | Integer modelling tricks                              | 54         |
| 5.2      | Solution algorithm: Branch & Bound                    | 56         |
| 5.2.1    | Making solvable models                                | 58         |
| 5.3      | Solution hardness                                     | 59         |
| <b>6</b> | <b>Mixed Integer Programming problems</b>             | <b>61</b>  |
| 6.1      | Young Couple <sup>4</sup>                             | 62         |
| 6.2      | Startup Fund                                          | 63         |
| 6.3      | Logic                                                 | 65         |
| 6.4      | Stamps                                                | 66         |
| 6.5      | Micro brewery 2                                       | 68         |
| 6.6      | Santa's Workshop Tour 2019 <sup>5</sup>               | 70         |
| 6.7      | Factory Planning 2 <sup>6</sup>                       | 73         |
| 6.8      | Student Teacher Meetings                              | 74         |
| 6.9      | Workplan for Teaching Assistants                      | 76         |
| 6.10     | Scrap Removal                                         | 79         |
| 6.11     | Groceries                                             | 80         |
| 6.12     | Tennis                                                | 84         |
| 6.13     | Aircraft Landing                                      | 86         |
| 6.14     | Tariff Rates <sup>7</sup>                             | 88         |
| 6.15     | Railway Depot Shunting                                | 90         |
| 6.16     | Food Festival                                         | 93         |
| 6.17     | The Wedding Planner                                   | 95         |
| 6.18     | Hot Air                                               | 98         |
| 6.19     | Wind Farm Cable Routing                               | 103        |
| 6.20     | Wedding Planner with Math-Heuristics                  | 107        |
| 6.21     | Math-heuristics for Stamps                            | 108        |
| 6.22     | University Timetabling                                | 109        |
| <b>7</b> | <b>Multi-Objective Integer Programming</b>            | <b>115</b> |
| 7.0.1    | Multi-Objective Incredible Chairs Julia/JuMP program: | 116        |

---

<sup>3</sup>Taken from Williams: "Model Building in Mathematical Programming"

<sup>4</sup>Hillier and Lieberman: Introduction to Operations Research"

<sup>5</sup>This comes from the traditional Christmas optimization challenge, see:  
<https://www.kaggle.com/c/santa-workshop-tour-2019>

<sup>6</sup>Taken from Williams: "Model Building in Mathematical Programming"

<sup>7</sup>Taken from Williams: "Model Building in Mathematical Programming"

|           |                                                          |            |
|-----------|----------------------------------------------------------|------------|
| 7.1       | The importance of Multi-Objective models . . . . .       | 119        |
| 7.2       | Multi-objective theory . . . . .                         | 119        |
| 7.3       | Multi-objective optimization algorithms . . . . .        | 122        |
| 7.4       | The epsilon constraint algorithm . . . . .               | 122        |
| <b>8</b>  | <b>Multi-Objective optimization Programming problems</b> | <b>127</b> |
| 8.1       | Multi-objective Startup Fund . . . . .                   | 128        |
| 8.2       | Multi-objective Young Couple 2 . . . . .                 | 130        |
| 8.3       | The Multi-objective Wedding Planner . . . . .            | 131        |
| 8.4       | Multi-objective Aircraft Landing . . . . .               | 132        |
| 8.5       | $\epsilon$ -Constraint algorithm . . . . .               | 134        |
| <b>9</b>  | <b>Data</b>                                              | <b>135</b> |
| 9.1       | Direct data definition . . . . .                         | 135        |
| 9.2       | Including Julia data . . . . .                           | 136        |
| <b>10</b> | <b>Future reading</b>                                    | <b>139</b> |
| 10.1      | Classic OR problems . . . . .                            | 139        |
| 10.2      | Optimization hardness . . . . .                          | 140        |
|           | <b>Bibliography</b>                                      | <b>143</b> |



## 2 Installation

This book focuses on Mathematical Programming modelling, and we will use Julia and JuMP as the vehicle to implement the models. This is necessary, since these models cannot be solved analytically. We must emphasize that this book is **not** about Julia programming. For this, there are a number of other resources available at [www.julialang.org](http://www.julialang.org). JuMP is a software package which makes it easy to implement Mathematical Programming models in Julia. In this chapter, we will briefly describe how to install Julia, JuMP, selected solvers, and software editors.

### 2.1 Julia

Julia is a new programming language, initiated in 2009, which is "as fast as C and as easy as Python". Installing Julia is easy, simply download the correct version for your computer from <https://julialang.org/downloads/> and follow the instructions. In this course we will use version v1.8.3 of Julia, which is the newest version. If you have a slightly older version, e.g. v1.7, you do not have to reinstall Julia. Julia supports Windows, Mac, Linux, and FreeBSD.

### 2.2 JuMP

When Julia is installed, installing JuMP is easy: Start Julia in a command-line. In the command-line write: `import Pkg; Pkg.add("JuMP")`. Then the JuMP package will be automatically installed. This may take a few minutes.

### 2.3 Solvers

Given a Mathematical Programming model, either a Linear Programming (LP) or Mixed Integer Programming (MIP) solver is necessary. In this course our standard solver is HiGHS, which is open-source. If you already have Gurobi installed this can also be used.

The JuMP modelling package however supports a number of solvers, both commercial and open-source:

1. HiGHS: **open-source** solver, for both LP models and MIP models
2. Gurobi: Commercial industry leading solver, for both LP models and MIP models
3. Cplex: Commercial industry leading solver, for both LP models and MIP models
4. Clp: **open-source** solver for solving LP models. Not further developed
5. GLPK: **open-source** solver for solving both LP models and MIP models. Not further developed
6. Cbc: **open-source** solver for solving MIP models. Not further developed
7. Xpress: Commercial industry leading solver, for both LP models and MIP models

In this book, we recommend to use either HiGHS, Gurobi, or CPLEX. HiGHS is an opensource solver which is just as easy to install as JuMP and can be used for free. All the models in this book can be solved with the best open-source solver, HiGHS, if the model is implemented correctly. The two commercial solvers Gurobi and CPLEX are state-of-art solvers and can be obtained for academic usage for free, but they are harder to install and can then only be used for teaching and research. But they are significantly faster for most MIP models.

There is a longer list of supported solvers shown in the JuMP documentation <https://jump.dev/JuMP.jl/stable/installation/> and the list keeps growing. We have picked what we consider the best solvers for use in this book above. Furthermore, a number of the supported solvers are for other types of mathematical models not considered in this book: QP (Quadratic Programming), NLP (Non-Linear Programming), (MI)NLP (Mixed Integer Non-Linear Programming), SOCP (Second-Order Cone Programming), SDP (Semidefinite programming), and several others.

## 2.4 Editors

The Julia/JuMP models are in principle simple ASCII text files, and any editor can be used to manipulate them, even Windows Notepad can be used. The models can then be executed from the command-line. We recommend the use of Visual Studio Code, <https://code.visualstudio.com>.

Installing these environments, both of which are open-source, and connecting them to Julia can sometimes be tricky, depending on your computer setup.



## 2.5 Why Julia ?

The material in this book is part of the material used in our course "Mathematical Programming Modelling" (42112), taught at the Technical University of Denmark. This course about making LP models and MIP models has been taught at our university for more than 40 years, the last 18 years by us. For the entire period until 2018 the specialized Mathematical programming language GAMS [www.gams.com](http://www.gams.com) was used. Since 2019 we have used Julia/JuMP. Using Julia/JuMP comes with a number of advantages:

- The whole system is open-source, when using the HiGHS open-source solver, i.e. free of charge to use.
- Integration with other programs is simpler, because these can simply be programmed in Julia.
- Like GAMS, multiple different solvers can be applied, and easily be switched between different solvers.

Our experience with Julia/JuMP since 2019 has been so good that we consider the days of dedicated Mathematical Programming languages like GAMS, AIMMS, AMPL, MPL to be over. Julia/JuMP is now the only programming language used in the courses at the Operations Research section, at the Department of Management Engineering. It is being used in all our 10+ courses on Operations Research for, e.g., meta-heuristics, Dantzig-Wolfe decomposition, Benders Decomposition and Network optimization.



## 3 Linear Programming

LP is a classic technique in Operations Research and was invented independently several times during World War II. With the invention of the Simplex algorithm, by George Dantzig (1947), the models could also be solved to optimality, i.e. one could find a best solution value to the the decision variables given an objective function and a set of constraints. The optimization algorithms for LP is not the topic of this book, in Chapter 10 we will give links to literature on this important topic. Here we focus on **modelling** problems with the LP model. We will now introduce LP models using the two first assignments from Chapter 4, "Incredible Chairs" and "Incredible Chairs II".

### 3.1 Incredible Chairs

The company Incredible Chairs produces two different types of chairs,  $A$  and  $B$ , where one unit (pallet of chairs) of Chair  $A$  can be sold for a profit of 4 and one unit of Chair  $B$  can be sold for a profit of 6. The chair require time on three different production lines to be produced where the lines represent different processes that the chairs go through. To produce one unit of a chair it must spent the necessary time on each line

1. Line 1 requires 2 hours to produce one unit of Chair  $A$ , and at most 14 hours can be used due to other production on the line.
2. Line 2 requires 3 hours to produce one unit of Chair  $B$ , and at most 15 hours can be used.
3. Line 3 requires 4 hours to produce one unit of Chair  $A$  and 3 hours to produce one unit of chair  $B$ , and at most 36 hours can be used.

Suppose you are the production planner at the company, how will you plan the production of the chairs? The first important thing to identify is what you are in control of. In other words, what are the decisions you can make as production planner? In this case, you can decide **how many** of each type of chair to produce. Since these quantities are unknown and must be decided, we define two **decision variables**,  $x_A \geq 0$  and  $x_B \geq 0$ , which will store the quantity of each type of chair. Notice that the variables have a domain, i.e. an interval of feasible values. Here, they are each constrained to take a

positive value. The job for you as the production planner is to find the best possible values for these two variables.

There are many possible solutions represented by the decision variables ( $x_A$  and  $x_B$ ). As the production planner, you are, however, not interested in a random production plan, stated in the variables  $x_A$  and  $x_B$ , but you want the **best feasible** production plan. To compare the different solutions, an **objective function**  $f(x_A, x_B)$  which quantifies the utility of the solution is used. For this example, the objective function measures the profit of the solution obtained by producing  $x_A$  and  $x_B$  chairs. The objective function can therefore be stated as:

$$\max f(x_A, x_B) = 4 \cdot x_A + 6 \cdot x_B$$

Not all solutions are possible; they have to be **feasible**. This means that the values assigned to the variables should satisfy the restrictions that are imposed by the production lines. In particular, for each production line, the planned production of chairs cannot exceed the resource limit available. On line 3, the sum of the required resources cannot exceed 36. Given the unknown values of the decision variables, we can now formulate an equation - a **constraint** - that limits the values of  $x_A$  and  $x_B$  to values which are possible given the limitations on production line 3:

$$4 \cdot x_A + 3 \cdot x_B \leq 36$$

Similarly, there are two other constraints imposed by lines 1&2. They are given below:

$$2 \cdot x_A \leq 14$$

$$3 \cdot x_B \leq 15$$

The complete LP model is therefore:

$$\begin{array}{llll} \text{Maximize.} & 4 \cdot x_A & + & 6 \cdot x_B \\ \text{Subject to :} & & & \\ & 2 \cdot x_A & & \leq 14 \\ & & 3 \cdot x_B & \leq 15 \\ & 4 \cdot x_A & + & 3 \cdot x_B \leq 36 \\ & x_A & & \geq 0 \\ & & x_B & \geq 0 \end{array}$$

LP models are **linear**, i.e., the only allowed calculations in objective function and constraints are multiplications between constants and decision variables, and the addition and subtraction of variables.

One of the reasons for the popularity of LP models is that today there are a number of efficient mathematical optimization algorithms - *solvers* - which can be used to **solve** (i.e., find the optimal solution to) the LP model. Solvers optimize the mathematical model and in doing so find the best possible feasible values for the decision variables (here  $x_A$  and  $x_B$ ). To solve the model, it must be loaded into a solver. In this book, we use the standard programming language Julia in combination with the specialized mathematical programming modelling package JuMP. The syntax is different to the above mathematical model but very similar. Below we list the compact version, in Julia/JuMP.

#### Compact IC model version:

---

```
1 using JuMP
2 using HiGHS
3 IC = Model(HiGHS.Optimizer)
4
5 @variable(IC, xA >= 0)
6 @variable(IC, xB >= 0)
7 @objective(IC, Max, 4*xA+6*xB)
8 @constraint(IC, 2*xA <= 14)
9 @constraint(IC, 3*xB <= 15)
10 @constraint(IC, 4*xA+3*xB <= 36)
11 print(IC)
12 optimize!(IC)
13 println("Termination status: $(termination_status(IC))")
14 if termination_status(IC) == MOI.OPTIMAL
15     println("Optimal objective value: $(objective_value(IC))")
16     println("xA: ", value(xA))
17     println("xB: ", value(xB))
18 else
19     println("No optimal solution available")
20 end
```

---

It should be clear that despite the different syntax there is a one-to-one correspondence between the LP model given above and the Julia/JuMP model on lines 5 to 10. Notice that the LP model assumes that chairs can be produced in any number, including fractional amounts. In fact, the optimal production plan is to produce 5.25 of chair A and 5 of chair B. If we impose the requirement that  $x_A \in \mathbb{Z}^+$ , i.e. that only **integer** amounts of chairs can be produced, the model becomes a MIP model. This is described further

in Chapter 5.

When we execute the model in Julia, we get the following printout:

```
Max 4 xA + 6 xB
Subject to
  2 xA <= 14.0
  3 xB <= 15.0
  4 xA + 3 xB <= 36.0
  xA >= 0.0
  xB >= 0.0
Presolving model
1 rows, 2 cols, 2 nonzeros
1 rows, 2 cols, 2 nonzeros
Presolve : Reductions: rows 1(-2); columns 2(-0); elements 2(-2)
Solving the presolved LP
Using EKK dual simplex solver - serial
  Iteration      Objective      Infeasibilities num(sum)
          0      0.0000000000e+00 Ph1: 0(0) 0s
          1     -5.1000000000e+01 Pr: 0(0) 0s
Solving the original LP from the solution after postsolve
Model   status      : Optimal
Simplex iterations: 1
Objective value      :  5.1000000000e+01
HiGHS run time       :           0.00
Termination status: OPTIMAL
Optimal objective value: 51.0
xA: 5.25
xB: 5.0
```

The model is output in first seven lines as per the instruction line 11 of the Julia/JuMP program. This can be beneficial when debugging a model, but is only ever relevant for relatively small models. The following fourteen lines of output comes from the HiGHS solver. The program then prints out that the optimal solution has been found (line 13 in the program) and then finishes by printing the optimal, maximal profit (51.0) (line 15), and the optimal values of the two decision variables,  $xA = 5.25$  and  $xB = 5.0$  (line 16 and line 17).

## 3.2 Incredible Chairs 2

What if the company were not producing two different chairs on three production lines, but say ten different chairs on five different production lines? If the same approach were used, the size of the LP model would increase, and it would be quite tedious to input using the direct approach above. In this case, we can formulate a more abstract model. Given a set of chairs  $C$ , indexed by  $c$ , and a set of production lines  $P$ , indexed by  $p$ , we can define a (one-dimensional) vector  $Profit_c$ , which states the profit for each chair  $c \in C$ , a (one-dimensional) production capacity vector  $Capacity_p$ , which states the production capacity of each production line  $p \in P$ , and a (two-dimensional) matrix  $RecourseUsage_{p,c}$  that states how much of the available resource on production line  $p$  is needed for production of one chair  $c$ .

This gives the following LP model:

$$\begin{array}{ll}
 \text{Maximize.} & \sum_{c \in C} Profit_c \cdot x_c \\
 \text{Subject to :} & \\
 & \sum_{c \in C} RecourseUsage_{p,c} \cdot x_c \leq Capacity_p \quad \forall p \in P \\
 & x_c \geq 0 \quad \forall c \in C
 \end{array}$$

Note the use of the symbol  $\forall$  meaning "for all", which means that the statement in the constraint needs to hold for all production lines or chairs.

The above model is a very **generic** (chair) production model for profit maximization for any number of chairs and any number of production lines. In fact, any (simple) profit maximizing production problem with a number of products and production resources can be modelled in this way. In the remainder of this book, we will consider two types of variables: **direct variables** and **abstract variables**. The above model has 10 ( $x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, x_9, x_{10}$ ) direct variables, but only 1 abstract variable ( $x_c$ ). Correspondingly, we will consider two types of constraints, **direct constraints** and **abstract constraints**. The above model has 5 direct constraints but only 1 abstract constraint. In general, we measure the **complexity** of a model in the number of abstract variables and constraints, and the **size** of the model is measured in the number of direct variables and direct constraints.

The corresponding Julia/JuMP model can now be written as:

---

```

1 IC = Model(HiGHS.Optimizer)
2
3 @variable(IC,x[c=1:C] >=0 )
4
```

```

5 @objective(IC, Max, sum( Profit[c]*x[c] for c=1:C)
6
7 @constraint(IC, [p=1:P],
8 sum( RecourseUsage[p,c] * x[c] for c=1:C) <= Capacity[p])
9
10 optimize!(IC)

```

---

Notice that the program actually becomes smaller. Also there is a one-to-one correspondence between the individual elements.

- The definition of the variable  $x_c \geq \forall c$  is now given on one line, line 3:

```
@variable(IC,x[c=1:C] >=0 )
```

- The sum operator  $\sum$ , over an index of a set, is used both in the objective function and in the constraint:

- The objective function, line 5,  $\sum_{c \in C} Profit_c \cdot x_c$  is written as:

```
@objective(IC, Max, sum( Profit[c]*x[c] for c=1:C))
```

- The constraint:  $\sum_{c \in C} RecourseUsage_{p,c} \cdot x_c \leq Capacity_p \forall p \in P$  is written in 7 and 8 as:

```
@constraint(IC, [p=1:P],
sum( RecourseUsage[p,c] * x[c] for c=1:C) <= Capacity[p])
```

- The repetition of a constraint for all values of an index does not require a formal for-loop but can be parsed to the `@constraint` macro as demonstrated above where  $\forall p$  is simply adding `[p=1:P]` before the constraint statement.

In line 1, the data-structure in which the variables, objective function, and constraints are contained is constructed, named IC. Here the solver needed to solve the model is also defined.

```
IC = Model(HiGHS.Optimizer)
```

When the model is considered to be complete, it is transferred to the solver, which finds the optimal solution to the model. This happens in line 10:



`optimize!(IC)`

When the `optimize!` command is given, the full model is transferred to the solver, HiGHS in this case.

The `Model` construct, and the different model structures `@variable`, `@objective`, and `optimize!` are part of the JuMP package.

Setting up the data-structures `Profit`, `Capacity`, and `RecourseUsage` in Julia is described in Chapter 9. Below we present full Julia/JuMP program with the correct numbers and data-sets for the Incredible Chairs 2 assignment. This format is used in all of the assignments in the book.

### Standard Incredible Chairs 2 model version:

---

```
1 *****
2 # Incredibly Chairs 2 assignment, Simple LP
3 using JuMP
4 using HiGHS
5 *****
6
7 *****
8 # Data
9 Chairs=["A", "B", "C", "D", "E", "F", "G", "H", "I", "J"]
10 C=length(Chairs)
11 ProductionLines=[1, 2, 3, 4, 5]
12 P=length(ProductionLines)
13
14 Profit=[6, 5, 9, 5, 6, 3, 4, 7, 4, 3] # profit for each chair
15 Capacity=[47, 19, 36, 13, 46] # production line capacity
16 RecourseUsage=[ # RecourseUsage[p,c] res use for chair c on prod. line p
17 6 4 2 3 1 10 2 9 3 5;
18 5 6 1 1 7 2 9 1 8 6;
19 8 10 7 2 9 6 9 6 5 6;
20 8 4 8 10 5 4 1 5 3 5;
21 1 4 7 2 4 1 2 3 10 1]
22 *****
23
24 *****
25 # Model
26 IC2 = Model(HiGHS.Optimizer)
27
28 @variable(IC2,x[c=1:C]>=0) # x[c] is the number of chairs produced
```

```

29
30 # Objective is to maximize profit
31 @objective(IC2, Max, sum( Profit[c]*x[c] for c=1:C ) )
32
33 # Constraints ensuring production line capacity is not exceeded
34 @constraint(IC2, [p=1:P],
35               sum( RecourseUsage[p,c]*x[c] for c=1:C) <= Capacity[p]
36               )
37 *****
38
39 *****
40 # Solve
41 optimize!(IC2)
42 println("Termination status: $(termination_status(IC2))")
43 *****
44
45 *****
46 if termination_status(IC2) == MOI.OPTIMAL
47     println("Optimal objective value: \$(objective_value(IC2))")
48     for c=1:C
49         if value(x[c])>0.001 # only print the chairs actually produced
50             println("No of chairs of type ", Chairs[c], " produced: ",
51                     ↪ value(x[c]))
52         end
53     end
54 else
55     # model is in-feasible or un-bounded
56     println("No optimal solution available")
57 end
58 *****

```

The whole Julia/JuMP program consists of 5 overall parts, divided by the comment lines of asterisk. `#` is used for comments. The program parts are:

- The initial section, 1-5. Here the program is named and the `Using` keyword is used to load the different packages.
- The data section, 7-22. Here the data needed for the model is defined. For a description of how to define data, see Chapter 9
- The model section, 24-37. Here the decision variables, objective function and constraints are defined.
- The solve section, 39-43. Here the model is transferred to the solver and solved.

- The result section, 45-57. The results from the solver are printed.

## 3.3 Good modelling practices and debugging

There are a number of recommended practices in the above example:

- Use comments extensively; it makes the programs more readable. The character `#` makes all text afterwards on that line a comment, which is shown in green above.
- Given a **set** of items, define them as a one-dimensional vector (Chairs line 9) **and** define the length of the vector, as an upper-case letter (C line 10).
- Choose your index names carefully! As an example,  $c$  for chairs and  $C$  for the number of chairs. This is particularly important since there is no type-checking of sets in Julia/JuMP; The instruction `RecourseUsage[c,p]` is a valid instruction but will lead to an error, it should have been `RecourseUsage[p,c]`.

### 3.3.1 Debugging

Errors can be one source of frustration when implementing mathematical models. It is important to have strategies to resolve them when they do occur. To provide an example of how one might resolve an error, we deliberately introduce a programming error in the above program and change line 41 to:

```
optimize!(ICC)
```

When calling Julia, we get the following error report:

```
ERROR: LoadError: UndefVarError: ICC not defined
Stacktrace:
 [1] top-level scope
      @ ~/Desktop/IncredibleChairs.jl:41
in expression starting at /Users/thomasstidsen/Desktop/IncredibleChairs.jl:37
```

Error messages may not be very descriptive at times, and very often the best clue is to look for the line number of the error (here 41). The programs that are considered in this book are all of a relatively limited size, usually less than 200 lines of code. However, constraints that conflict with each other can lead to errors which are very difficult to spot.

Assuming that there are no Julia programming errors (as above), the objective of the model can be:

- An optimal and correct solution. We check if an optimal solution is found in line 46. If yes, the solution is printed out in lines 45-57. Notice the use of the two JuMP functions:
  1. `objective_value(IC2)` : After optimization, if there is an optimal result, we can obtain the optimal value using this function on the model which we here named IC2.
  2. `value(x[c])` : After optimization, if an optimal (or at least feasible) solution was found, then we can obtain the value of the decision variables for this solution i.e. the optimal number of chairs which should be produced.
- An optimal but incorrect solution: In all of the assignments in this book, the optimal objective value is given. If your model does **not** give this value, then there is definitely an error in your model. If your model **does** get the same optimal value, then your model **may** be correct, but this is not a guarantee.
- Unbounded: If the test in line 46 evaluates to false, the model may be unbounded. This means that the objective tends to  $+\infty$  ( for a maximization problem) or  $-\infty$  (for a minimization problem). In both cases, there is an error in your model (this is not a lost opportunity to earn an infinite amount of money). Often this error comes down to two possibilities: You have forgotten to set the domain of your variables, e.g.  $x_i \geq 0$  or you have inadvertently omitted a constraint that limits values of one or more of your decision variables. In the above example, if we add an extra chair, chair no. 11, and define the resource usage to be 0 for all production lines (e.g. it could be produced on a new line we forget to include), and there is a positive profit value, i.e. `Profig[11]>0`, **then** the solver will try to produce an infinite number of chairs of type 11.
- Infeasible: This means that there is no feasible solution to the model. This can happen; however, in this book all assignments have a feasible, optimal solution. In the above example, if just one of the capacity values take a negative value e.g. `Capacity[1]=-47`, the whole model becomes infeasible. The simplest way to debug this case is to insert sets of constraints one-by-one to see at which point feasibility of the problem is lost.

## 3.4 Conditional sums and constraints

Sometimes it is necessary to make conditional constraints or to use sum over a subset of elements in the objective function and/or constraints. This is easy to implement in Julia/JuMP:

- **Conditional constraint:**  $\sum_k x_{i,j,k} \leq 1 \quad \forall i, j | i \neq j$

```
@constraint(M, [i=1:I,j=1:J;i!=j], sum( x[i,j,k] for k=1:K) <= 1)
```

Notice that here the ";" defines the beginning of the condition in the "for all" part of the constraint.

- **Conditional sum:**  $\sum_{i,j|i \neq j} x_{i,j}$

```
sum( x[i,j] for i=1:I,j=1:J if i !=j )
```

Notice that here "if" defines the beginning of the condition in the sum.

This kind of conditional statement is necessary in some of the assignments.

## 3.5 Balance constraints

A very common constraint type used in LP and MIP models is the so-called **balance constraint**. Such a constraint connects two different sets of variables and ensures that "what was before" and "what is after" is the same such that everything is accounted for or in other words balanced.

Some examples of balance constraints that you will encounter throughout the exercises include the following:

- *Product flows:* A storage facility where products are first transported from factories into the storage facility and then subsequently transported from the storage facility to the customers. Since there is no production at the storage facility nor possibility for long term storage, then the quantity of product delivered from the different factories to the storage facility should match precisely the quantity of product delivered from the storage facility to the customers. This is an example of flow conservation which is very typical to encounter in network problems.
- *Inventory management of a certain product:* For a given time horizon, and an initial storage level of the product, the quantity of product on hand at the end of the time horizon must be the initial storage level adjusted by any new production (increasing the quantity on hand) and the amount of product that is sold or released within the time horizon (decreasing the quantity on hand).

The inventory management example will be a returning example in this course. Note that it requires one to relate a given time slot (index) with its previous time slot (index) and with some pre-defined boundary conditions on where to start and how to end at the edges of the considered time horizon. What is the time slot before the the first time

slot when that does not exist? In Julia, the in-line if statement can be used for cases like that: `(A ? B : C)` is an if statement. It evaluates whether `A` is true or false. If `A` is true, then it uses `B` and if `A` is false, then it uses `C`. Therefore, a statement like `(t>1 ? <normal case> : <what to do if this is first time-slot>)` can be used.

## 3.6 Relaxation and restriction of linear programs

Consider an LP model. By changing its set of constraints, one can either **relax** the model or **constrain** the model further.

**Relaxation:** If one (or more) of the constraints is removed from the model, then the model has been *relaxed*. The resulting model is termed a *relaxation*. Relaxations lead to optimal objective values that are at least as good as the original model (i.e. an increased value if maximizing and a decreased value if minimizing). The reason for this is that all of the previous solutions are still feasible, but there might also be new solutions that are now feasible due to the omitted constraints.

**Restriction:** If one (or more) constraint(s) is added to the model, then the model has fewer feasible solutions. Constraining a model more leads to (possibly) worse optimal objective values (i.e., a decreased value if maximizing and an increased value if minimizing). The reason for this is that some of the previously feasible solutions may now be infeasible given the newly added constraints.

## 3.7 Solver parameters

It is possible to transfer parameters to the solver via Julia/JuMP. This is often not so important for LP models since these are often solved quickly. When MIP models are solved, however, it may be interesting to set, e.g., a time limit or an acceptable optimality gap. Below we provide examples of how to set the values of some useful solver parameters. In the examples, `M` refers to the model name.

- Set a time limit for the solver. This may mean that the solver is forced to terminate after a specific time. The solver may not find the optimal solution or even find a feasible solution. Below, we demonstrate how the time limit is set to 300 sec.

```
set_time_limit_sec(M, 300.0)
```

- The printout from the solver can be suppressed with the `set_silent` function:

```
set_silent(M)
```

- Set an acceptable gap, i.e. a tolerance for how much below, what might be the optimal solution value, we allow our solution to be to still be considered optimal and stop solving further. This option becomes critical when working with MIP models. This parameter depends on which solver is used:

- Setting a gap of 10 % in HiGHS:

```
set_optimizer_attribute(M, "mip_rel_gap", 0.10)
```

- Setting a gap of 10 % in Gurobi:

```
set_optimizer_attribute(M, "MIPGap", 0.10)
```

- Setting a gap of 10 % in Cplex:

```
set_optimizer_attribute(M, "CPXPARAM_TimeLimit", 0.10)
```

## 3.8 Julia and JuMP structure

It is important to understand the differences and the relationships between Julia, JuMP, and the solver. In this section, we will briefly describe this.

### 3.8.1 Julia

Julia is a fully-fledged programming language that is used in many different application areas. In this book, we will only use a small subset of the Julia constructs.

### 3.8.2 JuMP

JuMP is a package for Julia and is specialized for creating Mathematical Programming models. In this book, we will only consider LP models and MIP models. JuMP can, however, be used for a number of other model types. JuMP extends Julia with some new constructs, created as macros. Consider the following example taken from Incredibly Chairs, see Chapter 3.1.

- The model statement:

```
IC = Model(HiGHS.Optimizer)
```

The model statement is used to define model.

- The variable statement:

```
@variable(IC, xA >= 0)
```

The variable statement is used to define a variable for a given model, in this case IC.

- The objective statement:

```
@objective(IC, Max, 4*xA+6*xB)
```

The objective statement is used to define an objective function for a given model, in this case IC.

- The constraint statement:

```
@constraint(IC, 4*xA+3*xB <= 36)}
```

The constraint statement is used to define one (or more constraints) for a mathematical programming model, in this case IC.

- The optimize statement:

```
optimize!(IC)
```

The optimize statement is used to transfer the model to the **solver**.

- The value statement:

```
value(xA)
```

This function is used to retrieve the value of a JuMP variable in the solution found by the solver. This function can only be used **after** the optimize! statement.

There are a number of important details:

- The first statement is always the model statement.
- The order of variable, constraint and objective statements does not matter. In this book, we will always first write the variables, then the objective function and finally the constraints



- There can be multiple variables, of different names, and multiple different constraints, but only one objective function. If several objective statements are issued, only the last before the optimize statement is used.
- The optimize statement builds the model in a solver-readable form and transfer the control to the solver.
- The variables of a JuMP model do **not** hold a value until after the optimize statement, The solver assigns the values, which can then be retrieved using the value statement. For this reason, the JuMP variables cannot be used in Julia constructs prior to the optimize statement.

### 3.8.3 Solver

One of the major advantages of using JuMP is that one can easily toggle between different solvers for the same model. The only changes necessary are in using statement at the top of the program and also in the model construction statement. JuMP supports a number of solvers for both LP models and MIP models, see the list of solvers we recommend in Section 2.3.

All models in this book can be solved in reasonable time using the opensource HiGHS solver. The two leading solver commercial solvers Gurobi and Cplex are much faster, in particular for MIP models, but they are relatively expensive. Academics and students can usually get access to an academic license for the solvers, but this requires a separate installation. The opensource solvers are, however, very simple to install in Julia, using the package system.

### 3.8.4 Structure

We are now ready to briefly describe the interactions of a model in Julia/JuMP with the solver, see Figure 3.1. The lowest level, i.e., the green part, corresponds to the Julia code part. When the program is executed, the initial part and any data will be initialized in the Julia program. This only affects the Julia part. The model is then built using the JuMP package. This is essentially an interface between the Julia program and the solvers (of which three are indicated in the figure). The JuMP package provides the model function and the @variable, @objective, and @constraint commands. These commands are used to build the LP model or MIP model. When the optimize statement is executed, the constructed JuMP model is transferred to the solver, which will then start to solve the model. When the solver terminates, the optimal objective value and corresponding solution is available via JuMP and can be queried from the Julia part using the respective functions `objective_value function()` and the `value()`.

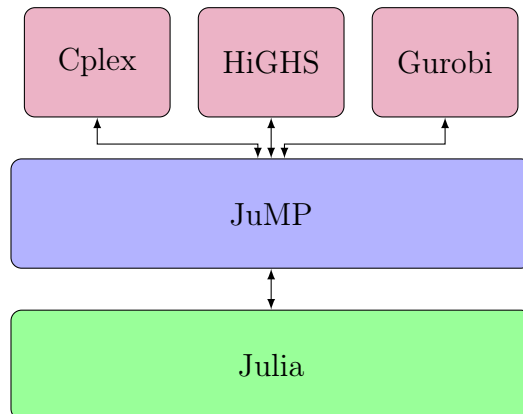


Figure 3.1: Julia/JuMP and Solver Structure

An important point is that the values of the JuMP variables, defined with the `@variable` statement, **cannot be used as variables in the Julia program !**. This is because, the value is set **only** by the solver and it can afterwards only be queried, not altered, using the `value()` function. We illustrate this with two deliberate errors in the Julia/JuMP program below (both will lead to errors in the compilation):

- In line 10, we attempt to make an in-line conditional statement in a constraint using the value of the JuMP variable `xA`. This is wrong for two reasons:
  - The `xA` JuMP variable does not have a value before the `optimize!()` statement.
  - Even if it had a value, the `value()` function is needed to extract this value
- In line 13, the value of the `xA` JuMP variable is used in an if statement, which is an error for the same reasons.

In line 21, the `value()` statement is used correctly to obtain the optimal value of the `xA` JuMP variable.

---

```

1 using JuMP
2 using HiGHS
3 IC = Model(HiGHS.Optimizer)
4
5 @variable(IC, xA >= 0)
6 @variable(IC, xB >= 0)
7 @objective(IC, Max, 4*xA + 6*xB)
8 @constraint(IC, 2*xA <= 14)
9 @constraint(IC, 3*xB <= 15)

```

```
10 @constraint(IC,4*xA+3*xB <= 36 + $(xA>2 ? 10 : 0 ) ) # <<<<< error
11 print(IC)
12
13 if xA>3 # <<<<< error
14     println("High production of chair A")
15 end
16
17 optimize!(IC)
18 println("Termination status: $(termination_status(IC))")
19 if termination_status(IC) == MOI.OPTIMAL
20     println("Optimal objective value: $(objective_value(IC))")
21     println("xA: ",value(xA)) # <<<<< correct
22     println("xB: ",value(xB))
23 else
24     println("No optimal solution available")
25 end
```

---



## 4 Linear Programming problems

This chapter is the first exercise chapter of the book. All the assignments in this chapter focus on LP.

## 4.1 Jewelry Production

Your aunt runs a small jewelry production company that produces relatively cheap necklaces (five different types), primarily for tourists. The profit for each necklace, price with material costs subtracted, is given in Table 4.1.

|            | Necklace type |    |    |    |    |
|------------|---------------|----|----|----|----|
|            | 1             | 2  | 3  | 4  | 5  |
| Profit (€) | 50            | 45 | 85 | 60 | 55 |

Table 4.1: Profit per necklace type (€)

Three machines are used to produce different jewelry components, which are then assembled by hand by two employees. Your aunt handles the machines herself; they are automatic but require some training to use. As they are automatic, all machines can run at the same. To produce the components for each necklace different amounts of time are required on each of the three machines. The machine time needed (in minutes) is given in Table 4.2. It is not necessary to produce components in a specific order.

| Machine | Necklace type |   |    |    |   |
|---------|---------------|---|----|----|---|
|         | 1             | 2 | 3  | 4  | 5 |
| 1       | 7             | 0 | 0  | 9  | 0 |
| 2       | 5             | 7 | 11 | 0  | 5 |
| 3       | 0             | 3 | 8  | 15 | 3 |

Table 4.2: The machine time needed for each necklace (minutes)

Additionally, each necklace must be assembled by one worker, and this takes some time (also measured in minutes). These times are given in Table 4.3. A full work day, both for the assembly and for each machine, is 7.5 hours.

|               | Necklace type |   |    |   |   |
|---------------|---------------|---|----|---|---|
|               | 1             | 2 | 3  | 4 | 5 |
| Assembly time | 12            | 3 | 11 | 9 | 6 |

Table 4.3: Necklace assembly time (in minutes)

**Assignment 1.1**

Formulate an LP that can be used to maximize the daily profit. You must determine the number of each type of necklace to produce, while taking into consideration the available machine and assembly hours.

*Optimal objective value for Product Mix: €5896.15.*

This is a simple example of a so-called *product mix* problem, a common application of LP.

Your aunt realize that she cannot sell all of the necklaces she makes, but estimates an average demand for each type. These values are given in Table 4.4

|                | Necklace type |    |    |    |    |
|----------------|---------------|----|----|----|----|
|                | 1             | 2  | 3  | 4  | 5  |
| Average Demand | 25            | 10 | 12 | 15 | 60 |

Table 4.4: Average demands

**Assignment 1.2**

Find a revised production plan that maximizes the daily profit. The production of the necklace types should not exceed the average demand values. Before you implement this constraint, try to reason if the additional constraints are going to increase or decrease the profit?

*Optimal objective value for jewellery production: €5643.18.*

Your aunt considers whether to hire another assembly worker. Is this a good idea ?

## 4.2 Micro brewery

You and a friend want to set up a microbrewery, both, to satisfy your own demand, but also to supply some nearby cafes with beer. The deal is that the cafes will get the beer relatively cheaply, but they will have to buy a certain amount (in liters) each month. The total collective monthly demand is given in Table 4.5:

|            | Months |     |     |     |     |     |     |     |     |     |     |     |
|------------|--------|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
|            | jan    | feb | mar | apr | may | jun | jul | aug | sep | oct | nov | dec |
| Demand (L) | 15     | 30  | 25  | 55  | 75  | 115 | 190 | 210 | 105 | 65  | 20  | 20  |

Table 4.5: Monthly demand for beer each month (in liters)

You set up the brewing system in your own room (you live at home with your parents) but can only brew at most 120 liters per month. This is a problem, given the demand, so you convince your mother that it is alright to store crates of beer in the basement. Her demand (to avoid excessive storage) is a storage cost of 1 € per liter of beer stored each month. Furthermore, you can store at most 200 liters of beer in the basement.

### Assignment 2.1

Formulate an LP that can determine the optimal beer production and storage strategy for one whole year, minimizing the storage costs. Assume you start with no beer in storage in January.

*Hint:* You need a beer production variable for each month and a storage variable for each month. The storage variable should reflect what is in storage *at the end of the month*. Then, you can make a *balance constraint*, relating what is in storage at the end of a month, with what was in storage at the end of the previous month. The previous month is the current month minus 1. *However*, if it is the *first* month (jan), then there is no storage. If your storage variable is  $y[m]$  (where  $m$  is the month index), you can add the previous month in Julia with the following code:

```
(m>1 ? y[m-1] : 0)
```

In general, this code  $(A ? B : C)$  is an if statement. It evaluates whether  $A$  is true or false. If  $A$  is true then it uses  $B$  and if  $A$  is false then it uses  $C$ .



### Assignment 2.2

Implement your LP model in Julia/JuMP.

*Optimal objective value for micro brewery problem: €560.0*

### 4.3 Blending <sup>1</sup>

Cooking oil is manufactured by refining raw oils and blending them together. The five different raw oils come in two categories, Vegetable and Non-vegetable. An overview is given in Table 4.6.

|                     |       |
|---------------------|-------|
| Vegetable oils:     | VEG 1 |
|                     | VEG 2 |
| Non-vegetable oils: | OIL 1 |
|                     | OIL 2 |
|                     | OIL 3 |

Table 4.6: The different oils

Vegetable oils and non-vegetable oils require different production lines for refining before they can be blended into the final product. In any month, it is not possible to refine more than 200 tons of vegetable oil and more than 250 tons of non-vegetable oils. There is no loss of weight in the refining process, and the cost of refining may be ignored.

There is a technological restriction of hardness in the final product. The final product must have a hardness value between 3 and 6, both values included, in the units used for measuring hardness. It is assumed that hardness blends linearly, i.e. if you use 2 tons of OIL 1 (hardness 2.0) and 3 tons of VEG2 (hardness 6.1), you get 5 tons of final product with a hardness of:  $(2 \cdot 2.0 + 3 \cdot 6.1) / (2 + 3) = 4.46$ . The costs (per ton) and hardness of the raw oils are given in Table 4.7:

|              | VEG 1 | VEG 2 | OIL 1 | OIL 2 | OIL 3 |
|--------------|-------|-------|-------|-------|-------|
| Cost (£/ton) | 110   | 120   | 130   | 110   | 115   |
| Hardness     | 8.8   | 6.1   | 2.0   | 4.2   | 5.0   |

Table 4.7: Cost and hardness of the oils

The final product sells for £150 per ton.

#### Assignment 3.1

How should the cooking oil be made in order to maximize the net profit? To find out, make a Julia/JuMP model to plan the production

<sup>1</sup>Taken from Williams: “Model Building in Mathematical Programming”

*Optimal objective value for Blending Problem: £17592.59*

*Tip:* If the solver complains that the problem is *non-linear*, you have a problem: The model has to be linear (variables may be subtracted/added and multiplied with parameters but not e.g. multiplied/divided by each other). Can you change the constraint, such that it becomes linear but still work in the same way?

This is another very common type of application of linear programming although, of course, practical problems will generally be much bigger.

## 4.4 Blending 2 <sup>3</sup>

This is an extension of the Blending assignment. Instead of planning the production for just one month, production must now be planned for the six months from January to June (including both months). For each month of production, the hardness requirement of the finished product has to be satisfied, as before. However, the cost for buying the raw oils now varies between months. Raw oil may be purchased for immediate refinement and used in that month's final product or stored (unrefined) to be used later. Prices for the raw oils in the different months are given below (in £/ton):

|          | VEG 1 | VEG 2 | OIL 1 | OIL 2 | OIL 3 |
|----------|-------|-------|-------|-------|-------|
| January  | 110   | 120   | 130   | 110   | 115   |
| February | 130   | 130   | 110   | 90    | 115   |
| March    | 110   | 140   | 130   | 100   | 95    |
| April    | 120   | 110   | 120   | 120   | 125   |
| May      | 100   | 120   | 150   | 110   | 105   |
| June     | 90    | 100   | 140   | 80    | 135   |

Table 4.8: Oil prices (£/ton)

The final product (still) sells at £150 per ton, and the refinement capacities are the same, for each month. It is possible to store up to 1000 tons of each raw oil for later use. The cost of storage for both vegetable or non-vegetable oil is £5 per ton per month. The final product cannot be stored. Nor can refined oils be stored. At the beginning of January there are 500 tons of each type of raw oil in storage. It is required that these stocks will also exist at the end of June.

### Assignment 4.1

What buying and manufacturing policy should be adopted in order to maximize profit? Formulate this as an LP and implement the model in Julia/JuMP.

*Optimal objective value for Blending 2: £107842.59*

If you get £82750.00, take a look at your refinement capacity constraints, do they limit the right variables?

---

<sup>3</sup>Taken from Williams: “Model Building in Mathematical Programming”

## 4.5 Factory Planning <sup>4</sup>

An engineering factory makes seven products (PROD 1 to PROD 7) that each require certain manufacturing processes (grinding, vertical drilling, horizontal drilling, boring, or planing). The factory has four grinders, two vertical drills, three horizontal drills, one borer, and one planer. Each product yields a certain amount of profit (selling price minus the cost of raw materials pr unit). Additionally, each product requires a certain amount of process time with each type of machine. Profits and process times are shown in Table 4.9. A dash indicates that the product does not require processing with that machine.

|                         |                     | PROD |      |      |       |      |      |      |
|-------------------------|---------------------|------|------|------|-------|------|------|------|
|                         |                     | 1    | 2    | 3    | 4     | 5    | 6    | 7    |
| Profit (£/unit)         |                     | 10   | 6    | 8    | 4     | 11   | 9    | 3    |
| Process<br>Time (hours) | Grinding            | 0.50 | 0.70 | -    | -     | 0.30 | 0.20 | 0.50 |
|                         | Vertical drilling   | 0.10 | 0.20 | -    | 0.30  | -    | 0.60 | -    |
|                         | Horizontal drilling | 0.20 | -    | 0.80 | -     | -    | -    | 0.60 |
|                         | Boring              | 0.05 | 0.03 | -    | 0.070 | 0.10 | -    | 0.08 |
|                         | Planing             | -    | -    | 0.01 | -     | 0.05 | -    | 0.05 |

Table 4.9: Product details

In the present month (January), and the five subsequent months, certain machines will be down for maintenance and cannot be used that month. A summary of the machines that will undergo maintenance is given in Table 4.10.

| Month    | Maintenance                     |
|----------|---------------------------------|
| January  | 1 grinder                       |
| February | 2 horizontal drills             |
| March    | 1 borer                         |
| April    | 1 vertical drill                |
| May      | 1 grinder and 1 vertical drill  |
| June     | 1 planer and 1 horizontal drill |

Table 4.10: Machine maintenance

The numbers of units of each product that can be sold in each month is limited by the market demand and are given in Table 4.11. Note that less than the full demand may

<sup>4</sup>Taken from Williams: “Model Building in Mathematical Programming”

be sold. Furthermore, it is possible to store up to 100 units of each product at a cost of £0.5 per unit per month. Products must be finished within one month, i.e. it is not possible to store half finished products between months, only finished products that have been processed fully on all machines they require. There are currently no units of any item in stock, but the factory would like to have 50 units of each type of product on hand at the end of June. The factory works a 6 day week with two shifts of 8 hours each day. You may assume that each month consists of 24 working days. No sequencing considerations need to be given when processing the products on the machines i.e. it is only necessary to consider the necessary time on the various machines not which order the processing takes place in.

|          | PROD |      |     |     |      |     |     |
|----------|------|------|-----|-----|------|-----|-----|
|          | 1    | 2    | 3   | 4   | 5    | 6   | 7   |
| January  | 500  | 1000 | 300 | 300 | 800  | 200 | 100 |
| February | 600  | 500  | 200 | 0   | 400  | 300 | 150 |
| March    | 300  | 600  | 0   | 0   | 500  | 400 | 100 |
| April    | 200  | 300  | 400 | 500 | 200  | 0   | 100 |
| May      | 0    | 100  | 500 | 100 | 1000 | 300 | 0   |
| June     | 500  | 500  | 100 | 300 | 1100 | 500 | 60  |

Table 4.11: Market limitations

### Assignment 5.1

Formulate an LP that can be used to determine when and what the factory should produce in order to maximize their profit. Implement your model in Julia/JuMP to find the optimal solution.

*Optimal objective value for the Factory Planning Problem: £93715.17 (ultimo storage)*

*Optimal objective value for the Factory Planning Problem: £93890.17 (primo storage)*

*Hint:* Think carefully about when you "count" your storage, it matters if you do it in the beginning of the month (*primo*) or at the end of the month (*ultimo*).

### Assignment 5.2

Estimate the value of acquiring new machines.

## 4.6 Project Scheduling

Your father is unhappy with the tool shed in the back garden of his house, see the picture. He wants a new one and has found a do-it-yourself kit tool shed from a hardware store. There is, however, a significant amount of work which has to be done. He splits the total project into tasks, some of which can be done by you, others must be done by specialists (electricians, plumbers etc.). He asks all contractors to commit to a day usage for the task. He offers you the first and last task and would like you to be in charge of the project. Given the list below, you need to make a *project plan*, i.e. when should each task be performed. To keep things simple, we will measure time in days. Notice that tasks can be carried out simultaneously, but some tasks require that other tasks have been completed before they can be started. As an example, the roof can only be installed after the walls have been erected.



Figure 4.1: The tool shed to be replaced

The list of all tasks to be completed is as follows:

1. Remove Stuff: Move the tools and other stuff out of the current shed and temporarily store them (duration 1 day)
2. Shed Demolition: Knock down the old shed (after Remove Stuff, duration 3 days)
3. Pipe Work: Install plumbing, there will be running water in the new shed (after Shed Demolition, duration 5 days)

4. Electric Work: There is electricity for tools and lights in the new shed (after Shed Demolition, duration 3 days)
5. Flagstones: The area around the shed should be covered in flagstones (after Pipe Work and Electric Work, duration 7 days)
6. Shed Foundations: Set the foundations for the new shed (after Pipe Work and Electric Work, duration 4 days)
7. Wood Frames: Erect the wooden walls of the shed (after Shed Foundations, duration 3 days)
8. Painting: Paint the walls (after Wood Frames, duration 1 day)
9. Roofing: Build the roof (after Wood Frames, duration 1 day)
10. Roofing Felt: Put roofing felt on the roof (after Roofing, duration 1 day)
11. Interior Installations: Install racks, lights etc. (after Roofing, duration 3 days)
12. Insert Stuff: Get the stuff out of the temporary storage and store them in the new shed (after Roofing, duration 1 day)
13. Hand over: Present the new shed to your father (After Flagstones, Painting, Roofing felt, Interior installations and Insert stuff, duration 0 days)

#### Assignment 6.1

Formulate the problem of minimizing the number of days needed to complete the shed as an LP. You may want to consider achieving this by minimizing the start time of the final task, "Hand over". The plan should take into account precedence restrictions of tasks. Implement and solve the model in Julia/JuMP.

*Optimal objective value for the Shed Project Scheduling : 20.0 days*

*Hint:* You can use conditional statements to only create constraints on the start time of tasks for which other tasks need to be fully completed before they may be started. See Section 3.4 for more on conditional constraints.

#### *Early finish*

Your father is not happy about the project plan, he needs the tool shed to be finished earlier. Looking at the plan, you contact all the contractors, who use more than one day for the job, and ask how much they can speed up the work, and how much overtime pay they need per speed-up day. This gives the information below.



- Shed Demolition: Can be reduced by up to 1 day at an overtime pay of 500 per day
- Pipe Work: Can be reduced by up to 3 days at an overtime pay of 2000 per day
- Electric Work: Can be reduced by up to 1 day at an overtime pay of 4000 per day
- Flag Stones: Can be reduced by up to 4 days at an overtime pay of 1000 per day
- Shed Foundations: Can be reduced by up to 2 days at an overtime pay of 1000 per day
- Wood Frames: Can be reduced by up to 2 days at an overtime pay of 1500 per day
- Interior Installations: Can be reduced 2 days at an overtime pay of 1500 per day

You realize that there is a problem. The completion time depends on the budget available for overtime, and the question your father has to answer is: How much is he prepared to pay for an earlier finish? Before you return to your father you first calculate how quickly the tool shed can be completed if there is an unlimited budget. For comparison, you also calculate the earliest completion times if you have a budget of 5000 and if you have a a budget of 10000.

### Assignment 6.2

Implement the Shed Project Scheduling in Julia/JuMP with the possibility to speed up tasks and minimize the number of days before hand over, taking into account an unlimited budget, a 5000 budget and a 10000 budget.

*Optimal solution for the Shed Project Scheduling, unlimited budget: 10.0 days*

*Optimal solution for the Shed Project Scheduling, budget of 5000 : 15.33 days*

*Optimal solution for the Shed Project Scheduling, budget of 10000 : 12.80 days*



## 5 Mixed Integer Programming

LP models are very useful, but there are many situations where the planning problem in question cannot be modelled sufficiently precisely. In many problems fractionality of the variables is problematic. Consider the Incredible Chairs problem again, now Incredible Chairs III: Since it is chairs that are produced, they need to be produced in **whole** numbers. Updating the model to achieve this in Julia/JuMP is simple (last line of model):

$$\begin{array}{llll} \text{Maximize.} & 4 \cdot x_A & + & 6 \cdot x_B \\ \text{Subject to :} & & & \\ & 2 \cdot x_A & & \leq 14 \\ & & 3 \cdot x_B & \leq 15 \\ & 4 \cdot x_A & + & 3 \cdot x_B \leq 36 \\ & x_A, x_B \in \mathbb{Z}^+ & & \end{array}$$

This is a Mixed Integer Programming (MIP) model where the variables are allowed to be restricted to only allowing **integer** values, as described above in the **domain constraints**. The below program will deliver the optimal **integer** solution. Only two things have been changed:

- In the definition of the variables, the variable is now declared to be integer, line 5 and line 6:

```
@variable(IC, xA >= 0, Int)
```

```
@variable(IC, xB >= 0, Int)
```

- When retrieving the value of the variable found by the solver, using the `value()` function, we **round** the resulting variable to become an integer type, line 16 and line 17:

```
println("xA: ", round{Int64}(value(xA)))
```

```
println("xB: ", round{Int64}(value(xB)))
```

Compact IC model version:

---

```
1 using JuMP
2 using HiGHS
3 IC = Model(HiGHS.Optimizer)
4
5 @variable(IC, xA >= 0, Int)
6 @variable(IC, xB >= 0, Int)
7 @objective(IC, Max, 4*xA+6*xB)
8 @constraint(IC, 2*xA <= 14)
9 @constraint(IC, 3*xB <= 15)
10 @constraint(IC, 4*xA+3*xB <= 36)
11 print(IC)
12 optimize!(IC)
13 println("Termination status: $(termination_status(IC))")
14 if termination_status(IC) == MOI.OPTIMAL
15     println("Optimal objective value: $(objective_value(IC))")
16     println("xA: ", round{Int64, value(xA)})
17     println("xB: ", round{Int64, value(xB)})
18 else
19     println("No optimal solution available")
20 end
```

---

From a modelling point of view, these changes are small and almost trivial. However, three important points need to be made, which are elaborated upon in the below sections:

1. The amount of important problems which can be better modelled using MIP models is **vast**. In the next section, we will briefly present a number of integer modelling tricks.
2. The required solution time of the solver may become critical: Finding the optimal solution for a MIP model becomes a much more challenging computational problem than for an LP model.
3. The type of algorithm necessary to find the optimal solution changes. To better understand the increased optimization complexity, we give a very brief introduction to the classic Branch & Bound algorithm.

### 5.1 Modelling power

At first glance, the addition of integer requirements for the variables does not seem like such a big change. However, many problems are **inherently** integer:

- *Chair production, or any type of production of integer amounts* - It is hard to sell fractional chairs.
- *Vehicle routing, e.g. trucks, ships, trains* - a fractional ship cannot sail.
- *Manpower planning* - planning with fractional workers is not a good option.
- *Timetabling* - the classes could be divided up, but what about the teacher?

These are just a few examples. It turns out that there are many cases where one or more variables needs to be integer ( $x \in \mathbb{Z}$ ) or binary ( $x \in \{0, 1\}$ ).

Generating a good integer solution from the Incredible chairs model is not complicated. The optimal value for Incredible chairs is 51 with the values  $x_A = 5.25$  and  $x_B = 5$ . Because all the coefficients of  $x_A$  are positive in the constraints and all the constraints are less-equal  $\leq$ ,  $x_A$  can simply be rounded down to  $x_A = 5$ , achieving an integer solution. This reduce the objective value to 50. But is this solution optimal? In the Incredible Chairs 3 assignment this is the topic.

In general, just rounding fractional variables is **not** guaranteed to reach an optimal solution, perhaps not even a feasible solution. In the Incredible Chairs case, the integer solution objective went from 51 to 50. In other cases, the changes in the objective can be far greater: Suppose a container ship has to be routed to pick up containers. If 3 containers have to be picked up from a port by a container ship with a capacity of 1000, an LP model will assign a  $\frac{3}{1000}$  ship to do the job. If what is minimized is cost, the objective can increase substantially by increasing the value from  $\frac{3}{1000}$  to 1.

An important special case of integer variables is **binary** variables i.e.  $x \in \{0, 1\}$ . This can model yes/no decisions, usually interpreting 1 as yes and 0 as no. A set containing  $C$  binary variable can easily be defined in Julia/JuMP as:

```
@variable(IC,x[c=1:C],Bin)
```

where IC is the name of the mathematical model. Notice that since a binary variable can **only** take the values 0 or 1, adding lower and upper bounds is not necessary. For integer variables however, upper bounds can be added in exactly the same way as for continuous variables.

```
@variable(IC,x[c=1:C],Int)
@variable(IC,x[c=1:C] >= 0,Int)
@variable(IC,0 <= x[c=1:C] <= 10,Int)
```

### 5.1.1 Integer modelling tricks

There are a number of standard modelling tricks which can be utilized when modelling with integer or binary decision variables. Here we briefly mention the most important constructions, and point to the assignments where they are needed.

**Fixed charge link:** Often a resource like a ship, depot, truck or plant, is represented by a binary variable (is it needed or not) or how many do we need. The cost of using the resource is then 0 if it is not used, but even the most limited use incurs a fixed cost. Mathematically speaking, this can be stated as:

$$h(x) = \begin{cases} 0, & x = 0, \\ f + px, & x > 0. \end{cases}$$

If we assume that there is an upperbound on  $x \leq B$ , then this can be modelled using binary decision variables in the following way:

$$\begin{aligned} h(x) &= fy + px, \\ x &\leq By, \\ y &\in \{0, 1\}. \end{aligned}$$

In Julia this would look like this:

---

```
1 @variable(IC, x >= 0)
2 @variable(IC, y, Bin)
3 @variable(IC, hx >= 0)
4
5 @constraint(IC, x <= B * y)
6 @constraint(IC, hx == f * y + p * x) # the resulting function
```

---

Such a fixed charge link could, e.g., correspond to the cost of transporting  $x$  containers on a container ship: There is an additional cost, due to extra fuel needed for the extra weight of the container, but the main cost is probably due to the need for a full ship, i.e. the  $y$  cost. This type of modelling is needed in Chair Logistics 2.

If the resource can be increased in fixed amounts, then the only change needed is to change the definition of the resource variable from binary to integer. This could correspond to, e.g., a number of container ships.

**Logic on binary variables**

Often binary variables can be used to formulate logical relations between different binary variables. Usually, a binary variable  $x = 1$  corresponds to true and  $x = 0$  corresponds to false. With these definitions, we can easily define the various logic operators:

**OR:** Given two binary variables  $x$  and  $y$ , define the  $v$  binary variable as  $x$  *OR*  $y$ :

$$\begin{aligned} v &\geq x \\ v &\geq y \\ v &\leq x + y \end{aligned}$$

**AND:** Given two binary variables  $x$  and  $y$ , define the  $v$  binary variable as  $x$  *AND*  $y$ :

$$\begin{aligned} v &\geq x + y - 1 \\ v &\leq x \\ v &\leq y \end{aligned}$$

**NOT:** Given one binary variable  $x$ , define the  $v$  binary variable as *NOT*  $x$ :

$$v = 1 - x$$

**Force selection:** Given a set of  $i$  items, select at least 3 of these:

$$\sum_i x_i \geq 3$$

**Limit selection:** Given a set of  $i$  items, select at most 4 of these:

$$\sum_i x_i \leq 4$$

A classical mistake in modelling is to try to model logic by using JuMP variables as Julia variables, as discussed in Section 3.8.4. Since this is not possible, when logic in the model needs to be modelled, it can be modeled as above. This is a requirement in the Startup Fund assignment 6.2.

**Binary multiplication:** All MIP model problems in this book can be modeled using **linear** models, i.e. where the variables are not multiplied. Given two binary variables  $x$  and  $y$  the multiplication of these can be formulated in a linear fashion, where the binary variable  $z$  is the result: (exactly as the  $x$  AND  $y$  operator)

$$\begin{aligned}z &\leq x \\z &\leq y \\z &\geq x + y - 1\end{aligned}$$

## 5.2 Solution algorithm: Branch & Bound

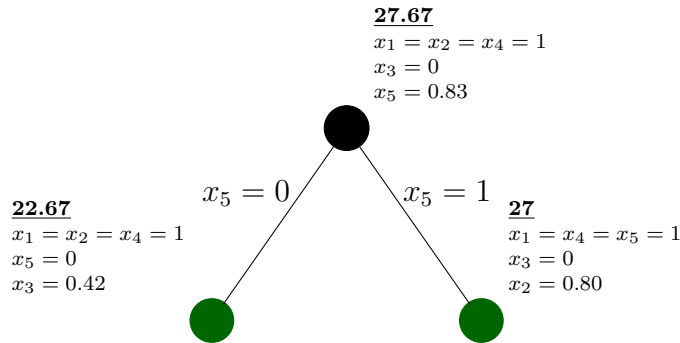
The focus in this book is on **modelling** LP and MIP models, not on the optimization algorithms needed to find the optimal solution to the models. In Chapter 8, we give references for further study, if you want to dive into this. It is, however, beneficial to have some intuition about the classical Branch & Bound algorithm for solving MIP models in order to better model different problems. In this section, we give a very simple and intuitive description of the basic approach.

The description of the algorithm is going to consider a very simple, small MIP model, the knapsack problem, see below:

$$\begin{array}{ll}\text{Maximize.} & 7 \cdot x_1 + 10 \cdot x_2 + 4 \cdot x_3 + 4 \cdot x_4 + 8 \cdot x_5 \\ \text{Subject to :} & \\ & 3 \cdot x_1 + 5 \cdot x_2 + 12 \cdot x_3 + 2 \cdot x_4 + 6 \cdot x_5 \leq 15 \\ & x_1, x_2, x_3, x_4, x_5 \in \{0, 1\}\end{array}$$

The knapsack problem contains 5 binary variables. In the Branch & Bound algorithm (B&B), first the LP version of the above problem, where the variables are **relaxed**, i.e.  $x_1, x_2, x_3, x_4, x_5 \in [0, 1]$ , is solved. This problem can easily be solved using an LP solver, reaching the optimal value 27.67, with the values:  $x_1 = x_2 = x_4 = 1$  and  $x_3 = 0$  and  $x_5 = 0.83$ . This solution is almost an integer solution, and the value 27.67 is a valid upper bound, i.e. we now know that **no** integer solutions with higher objective values exist. The B&B algorithm then selects the branching variable, i.e., selects one of the variables that is fractional and forces it to be either 0 or 1 by considering two new problems, where each of the new problems has an additional constraint. In one of the problems,  $x_5 = 0$ , and in the other  $x_5 = 1$ . This leads to two new solutions, shown as branches in a binary tree.



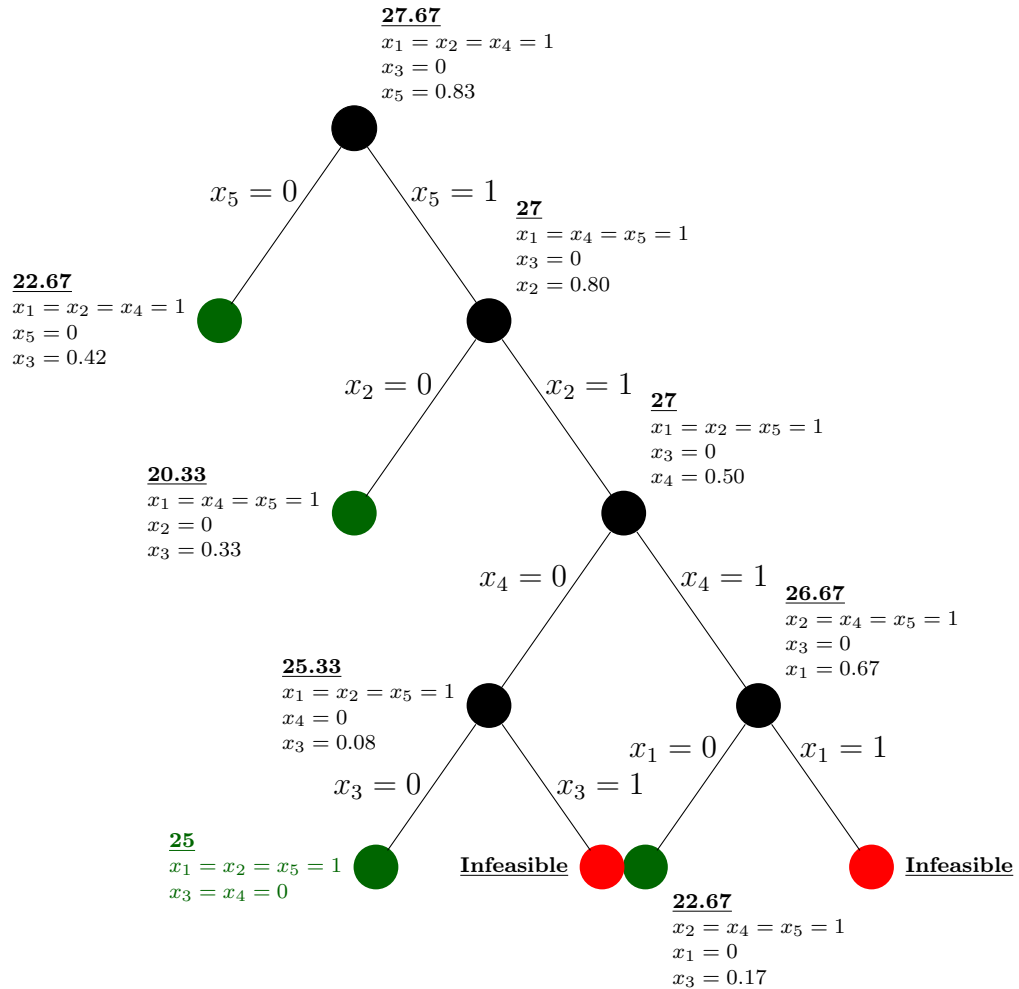


The black node is the so-called root solution. This is the LP relaxation we solved above and has an objective value of 27.67. The two new, green nodes reflect the impact of branching on variable  $x_5$ . The two problems are then solved giving solutions with objective values 22.67 ( $x_5 = 0$ ) and 27 ( $x_5 = 1$ ), respectively. The additional constraints are **restrictions** as mentioned in Section 3.6 and therefore the objective in both branches cannot have a higher value than 27.67. There are three possibilities for each of the new problems. These are:

- **The problem is infeasible:** It is not possible to find a (fractional or integer) solution, which is feasible. This means that any solution with more constraints will also be infeasible. Thus, we can **fathom** this node, i.e. not consider this node solution and any solution derived from it by branching. Fathoming is possible since it is guaranteed that no solution in the sub-tree of this node can improve the current best solution because all the solutions in the sub-tree will be infeasible.
- **The solution is integer:** This may be the optimal solution. If it is either the first solution found or better than the current best, then the solution is saved and termed the **incumbent**. It is the best integer solution found so far. All solutions in the sub-tree of this node can be ignored since they will be at best be as good as this solution.
- **The solution is fractional:** If the objective value is worse than the current incumbent, i.e., lower than the incumbent value for a maximization problem, then the node can be fathomed. Otherwise, a new fractional variable is selected and the process is repeated.

In this way, the Branch & Bound algorithm searches the binary search tree. If fathoming is not used and there are  $N$  binary variables,  $2^{N+1} - 1$  nodes need to be searched, i.e. LP optimized. The fully searched tree is shown below. Notice that due to fathoming, only 11 nodes are investigated. The full tree would contain 63 nodes. One of these leads to an integer solution with an objective value of 25, the optimal solution. The red nodes

are infeasible, and the remaining non-integer green nodes have optimal values less than 25 and therefore do not need to be explored.



Modern MIP solvers have evolved tremendously over the last 40 years. The description here is only meant to provide intuition behind the procedure.

### 5.2.1 Making solvable models

How hard a MIP model is, depends on both the size and formulation of the MIP model. More specifically, there are two aspects which are very important for the solvability of the model:

**The number of integer/binary variables:** The height of the Branch & Bound tree depends directly on the number of integer/binary variables. Two things can be done:

- Some integer variables will take integer values, even if not forced to. Then it should be formulated as a continuous variable.
- Integer variables for which it is possible to deduce their value should be fixed to this value, e.g.,  $x_{1,5} = 1$ . The easiest way to do this in Julia/JuMP is:

```
fix(x[1,5],1; force = true)
```

**MIP Gap:** The difference between the optimal value of the MIP model compared to the value of the optimal **relaxed** MIP model, where all the integer/binary variables have been converted to continuous variables. The size of this difference is critical. To find this gap, solve the first MIP model and compare to the optimal relaxed model which can relatively easily be found, using the following model:

```
relax_integrality(model)
optimize!(model)
```

If several different models are available for a problem, which often happens, we can compare the models and choose the best model, which is often the model with the fewest integer/binary variables and/or the lowest gap.

## 5.3 Solution hardness

The Branch & Bound algorithm for solving MIP models was invented by Ailsa H. Land and Alison G. Doig in 1960 Land and Doig (1960). Today MIP models are one of the most important tools applied in Operations Research. Importantly however, solving them is much harder than solving LP models. LP models can be solved with millions of variables; however, even MIP models with fewer than 100 variables may not be solvable to proven optimality. There are deep theoretical reasons for this difference, which we will briefly discuss in Chapter 10. However, since the invention of the Branch & Bound algorithm, researchers have continued to improve the approach and in particular in the last 30 years, the size of the MIP models which can be solved to optimality have improved tremendously. This has gradually enabled the use of MIP models to solve large real-world problems. A behaviour which is observed for all MIP models is that the solution time will grow **exponentially** with the size of MIP model, i.e. with the number of integer variables in the MIP model. In Figure 5.1 below, a MIP model for different

sized Travelling Salesman Problems (TSPs) is solved to optimality. The model of TSP tested is the model from the Grocery problem 6.11. Five different solvers are considered: HiGHS (light blue), CPLEX (grey), Gurobi (yellow), Cbc (dark blue) and GLPK (red) and they are all tested on 20 different random TSP problems for each size. All solvers show the same behaviour: Smaller models can be solved fast but when the size increase, at a certain **bending point**, the solution time suddenly increases dramatically. This kind of bending point exists for all MIP models and for all solvers, but as can be seen in Figure 5.1, it happens first for the GLPK solver, then the HiGHS solver, then the Cbc solver, then the CPLEX solver and finally the Gurobi solver. However, given a MIP model, if the model is of sufficiently small size, the solvers can handle the problem.

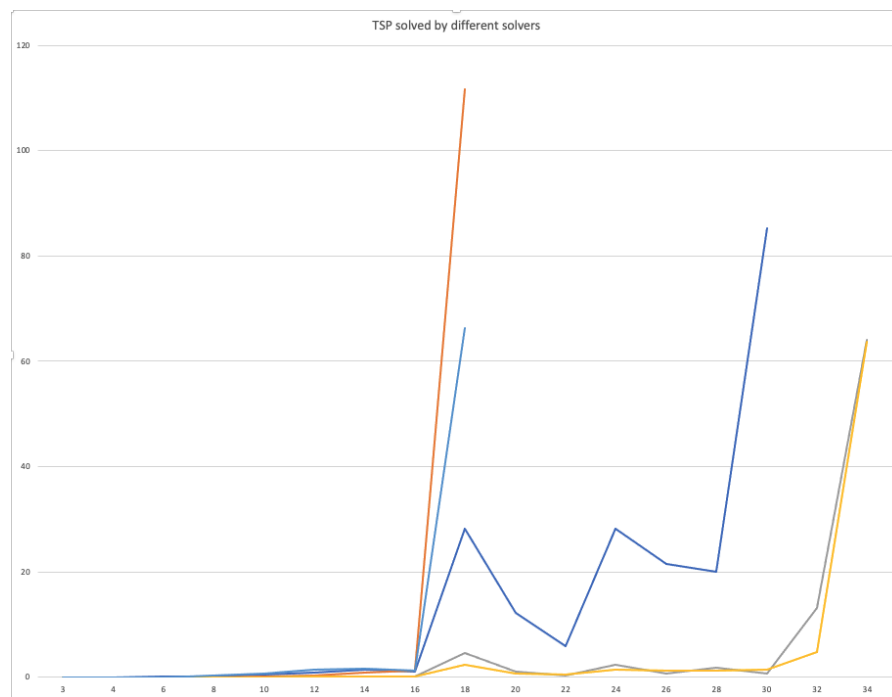


Figure 5.1: Average solution time of TSP with different solvers

Looking at the above graph, you may be wondering why we choose to prefer the HiGHS solver in this book. The answer is that in the future, the HiGHS solver is going to be further developed whereas the development of the Cbc solver seems stalled. Furthermore, the printout from the HiGHS solver is much better.

## 6 Mixed Integer Programming problems

This chapter contains MIP exercises. All of the assignments require the use of discrete decision variables and thus the formulation of MIP models. The problems considered appear in increasing order of difficulty.

## 6.1 Young Couple <sup>1</sup>

A young couple, Eve and Steven, want to divide their main household jobs (cleaning, cooking, dish-washing, and laundry) between themselves so that the total time they spend on household duties is kept to a minimum. Their efficiencies on these tasks differ, and the time each needs to perform the tasks is given in Table 6.1:

|        | Cleaning | Cooking | Dishwashing | Laundry |
|--------|----------|---------|-------------|---------|
| Eve    | 4.5      | 7.8     | 3.6         | 2.9     |
| Steven | 4.9      | 7.2     | 4.3         | 3.1     |

Table 6.1: Task times (hours)

### Assignment 1.1

Formulate a binary mixed integer model that minimizes the total number of required hours for household jobs and ensures that all jobs are done by someone. Implement and solve it in Julia/JuMP.

*Optimal objective value for Young Couple Problem: 18.2 hours*

### Assignment 1.2

Is the job assignment fair? Make a new program where Eve and Steven perform the same number of tasks.

*Optimal objective value for Young Couple Problem: 18.4 hours*

---

<sup>1</sup>Hillier and Lieberman: Introduction to Operations Research”

## 6.2 Startup Fund

A startup fund manager decides to use MIP models in order to optimize their profit. A new investment fund is just starting up, and €100 million is available for investments. The startup fund staff have identified nine possible investment opportunities in startup companies, the core information being the required capital to make the investment and the expected profit this will give at the end of the investment period.

|                  | Investment |    |    |    |    |    |    |    |    |
|------------------|------------|----|----|----|----|----|----|----|----|
|                  | 1          | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  |
| Estimated profit | 29         | 35 | 24 | 52 | 53 | 41 | 43 | 68 | 28 |
| Capital required | 17         | 25 | 19 | 25 | 28 | 23 | 29 | 31 | 18 |

Table 6.2: Investment details (millions of €)

Given an available budget of €100 million, the objective for the fund is to select the best possible set of investments such that the total estimated profit is maximized. Since these investment opportunities are in startup companies, it is only possible to invest or not invest, i.e. it is not possible to pay a fractional amount of the capital required, nor to pay more than the capital required. Money, which is not invested, is sent back to the investors in the Startup Fund and do not count towards the profit.

### Assignment 2.1

Formulate a binary integer program that can be used to maximize the total earned profit and implement it in Julia/JuMP.

*Optimal objective value for Investment Optimization Problem: €191 million*

### Assignment 2.2

After creating the first investment plan, the manager decides that investments in 1 and 5 are competing companies, so it is not possible to invest in both companies. Furthermore, the investments in 6 and 9 can only be performed if either investment 2 or investment 3 is done, since the developed technology in these is the basis of 6 and 9.

*Optimal objective value, maximizing the total earned revenue, for Investment Optimization Problem: €185 million*



## 6.3 Logic

This assignment should be done on paper. You have to relate three binary variables  $x$ ,  $y$  and  $v$ .

### Assignment 3.1: OR

Define (two or three) constraints, which relate the variables  $x$  and  $y$  to  $v$  such that if either  $x$  or  $y$  is 1 then  $v$  is forced to be 1. The truth table for the OR boolean operator is given below:

| $x$ | $y$ | $v$ |
|-----|-----|-----|
| 0   | 0   | 0   |
| 1   | 0   | 1   |
| 0   | 1   | 1   |
| 1   | 1   | 1   |

### Assignment 3.2: AND

Define (two or three) constraints such that if both  $x$  and  $y$  is 1 then  $v$  is forced to be 1. The truth table for the AND boolean operator is given below:

| $x$ | $y$ | $v$ |
|-----|-----|-----|
| 0   | 0   | 0   |
| 1   | 0   | 0   |
| 0   | 1   | 0   |
| 1   | 1   | 1   |

### Assignment 3.3: NOT

Define one constraint such that if  $x$  is 1 then  $v$  is forced to be 0 and if  $x$  is 0 then  $v$  is forced to be 1. The truth table for the NOT boolean operator is given below:

| $x$ | $v$ |
|-----|-----|
| 1   | 0   |
| 0   | 1   |

## 6.4 Stamps

Your uncle Archibald, whom you have not seen for 20 years, dies, and in his will, to your big amazement, lets you inherit his stamp collection. Your only problem is that you have absolutely no interest nor knowledge about stamps. Some of the countries from where the stamps originate, you have not even heard of!

Initially you consider whether to simply scrap the entire stamp collection of 100 stamps. Before you do so, you do, however, make a quick internet picture search for the stamps, and suddenly realize that at least some of the stamps are rather valuable! Therefore you instead decide to make some money, but you have zero knowledge about the stamp market. You consider to contact the local stamp collecting club but realize that your very limited knowledge makes it easy to trick you into a bad deal.

A friend comes up with a brilliant suggestion: Make an online auction! Since at least some of these stamps are rather valuable, it makes sense to do a little work. You and your friend make a simple website, where you put pictures of all of the 100 stamps, numbered from 1 to 100. You then send out emails to a number of stamp-collector clubs and ask for email-bids before a certain date.

A new problem then arises: Stamp collectors typically collect complete sets of stamps, e.g. all Danish stamps from 1977. Thus, if a stamp collector lacks two stamps to complete a collection, then he/she wants to buy *both* stamps or none of them. You receive 213 bids (in €) for different subsets of stamps from your set of 100 stamps. Each bidder wants *all* stamps in their bid at the price they offer. In other words, if they cannot get all of the stamps in the bid, then they will not have any.

The data for the problem can be downloaded from the book web-site <https://www.man.dtu.dk/MathProgrammingWithJulia>. The data, in Julia format, are:

- $BidPrice_b$ : The price offered in bid  $b$  in € for a set of stamps.
- $BidSets_{b,s}$ : Incidence matrix, where  $BidSets_{b,s} = 1$ , if bid  $b$  includes stamp  $s$  and 0 otherwise.

You can then either simply copy-paste the data into your program or you can use the `include(<file name>)` command.

**Assignment 4.1**

Formulate the problem of choosing which bids to accommodate, such that your profit is maximized, as an integer program and solve it using Julia/JuMP.

*Optimal objective value for the stamp problem: €11084*

You are very happy about the earnings but, to your surprise, not all stamps are sold! You discuss this with a friend, and he considers this a stupid loss, and blames the outcome on poor optimization. Hence, he suggests to *require* that all stamps are sold.

**Assignment 4.2**

Make the new model, what happens? Why? And is the suggestion good?

## 6.5 Micro brewery 2

Encouraged by your good results as a beer producer, you realize that if you offer a broader selection of beers, you may be able to sell more beers. So after some trial production, you decide to start the production of three types of beer: TSP-Stout, Knapsack-Dark, and Set-Partitioning-Light. After some tastings, the cafes order different amounts of beer. The total amounts ordered for each month and of each type are shown in Table 6.3:

|     | TSP-Stout | Knapsack-Dark | Set-Partitioning-Light |
|-----|-----------|---------------|------------------------|
| jan | 35        | 15            | 5                      |
| feb | 20        | 10            | 20                     |
| mar | 15        | 20            | 20                     |
| apr | 45        | 15            | 35                     |
| may | 25        | 15            | 35                     |
| jun | 65        | 55            | 80                     |
| jul | 40        | 90            | 60                     |
| aug | 50        | 80            | 30                     |
| sep | 35        | 25            | 35                     |
| oct | 85        | 45            | 20                     |
| nov | 50        | 5             | 20                     |
| dec | 55        | 30            | 40                     |

Table 6.3: Beer demand (litres)

You have the same brewing system, with a capacity of 120 litres, but you can only brew **one** type of beer per month! Since your mother wants to encourage your business, she allows you to store 300 litres of the beer types combined (i.e., the sum of stored beer over all three types can at most be 300 per month). She also reduces the storage cost to €0.1 per liter stored per month, for all beer types. You have produced an initial storage with 25 litres of TSP-Stout, 65 litres of Knapsack-Dark, and 75 litres of Set-Partitioning-Light.

### Assignment 5.1

Make a Mixed Integer Linear Programming model, which plans the production and storage for the whole year to meet demand optimally, minimizing the storage costs, and taking into account that only one type of beer can be produced per month. Implement the Microbrewery model in Julia/JuMP.

*Optimal objective value for Micro brewery 2 Problem: 192.5*

## 6.6 Santa's Workshop Tour 2019 <sup>2</sup>

*Hammers ring, are you listenin'*

*In the shop, toys are glistenin'*

*Should they see the sights?*

*There might be a fight*

### *Walkin' 'round the Workshop Wonderland*

*Families said, they want to see it*

*Santa said, he'd guarantee it*

*They pick a date*

*But they may have to wait*

## Walkin' 'round the Workshop Wonderland

*We told Santa that he was a madman*

*He just wants to make sure they all smile*

*He'll say "Are you flexible?", They'll say "Yeah man,*

*But can you help us make it worth our while?"*

*“Give them food, or sweater*

*the more they wait, the gifts get better"*

Please help us rank

*Or we'll break the bank!*

*Walkin' 'round the Workshop Wonderland*

Santa has exciting news! For 100 days before Christmas, he is opening up tour of his workshop. Because demand is so strong, and because Santa wanted to make things as fair as possible, he let each of the 5,000 families that will visit the workshop choose a list of dates they'd like to attend the workshop.

Now that all the families have sent Santa their preferences, he's realized it's impossible for everyone to get their top picks, so he's decided to provide extra perks for families that don't get their preferences. In addition, Santa's accounting department has told him that, depending on how families are scheduled, there may be some unexpected and hefty costs incurred.

Santa needs the help of the 42112 community to optimize which day each family should

<sup>2</sup>This comes from the traditional Christmas optimization challenge, see: <https://www.kaggle.com/c/santa-workshop-tour-2019>

get to go to the workshop in order to minimize any extra expenses that would cut into next years toy budget! Can you help Santa out?

Your job is to select the visit dates for each of the 5000 families. Each family consists of a number of persons and each family *HAS* to get a visit. Each family has given a list of wishes regarding visit dates, but these are NOT included in the data! Instead, these have been used to calculate DayVisitCosts for Santa Claus. This calculation is rather complex, but the calculations have been performed for you. See the details on the Kaggle website if your are interested. Each day there has to be at least 125 visitors and there can be at most 300 visitors.

When solving this problem, the difference between an open-source solver like HiGHS and a closed-source solver, e.g. Gurobi will be prominent. The free solvers cannot solve the full problem with 5000 families and 100 possible visit days days within a reasonable time. For this reason there are two data-sets for this problem: One with 5000 families and 100 days and a reduced data-set with 1000 families and 20 days. The data-set files are:

- `SantasWorkshopData_5000_100.jl`: The full data-set, with 5000 families and 100 days.
- `SantasWorkshopData_1000_20.jl`: The full data-set, with 1000 families and 20 days.

The data-set files defines 4 julia variables:

- $F$  : The number of families
- $D$  : The number of days for the visits (100 or 20)
- $FamilySize_f$  : The number of members of family  $f$
- $DayVisitCost_{f,d}$  : Cost of assigning family  $f$  to day  $d$ . These values include the cost of compensations for families who do not get assigned to a preferred day.

Also, the objective tolerance of the Gurobi solver may actually lead to a solution which is not precisely optimal, because the default objective tolerance is 0.01 % . This can be handled by setting the MipGap parameter for the solver, see Section 3.7

### Assignment 6.1

Formulate a mathematical model that can be used to optimally schedule the family visits at minimal cost. Implement your model in Julia/JuMP.

## 6 Mixed Integer Programming problems

---

*Optimal objective value for Santa's Workshop Tour 2019, 1000 families over 20 days:*  
164068

*Optimal objective value for Santa's Workshop Tour 2019, 5000 families over 100 days:*  
43622



## 6.7 Factory Planning 2 <sup>4</sup>

Instead of stipulating when each machine is down for maintenance in the factory planning problem, it is desired to find the best month for each machine to be down. Refer to Section 4.5 all previous information.

Except for the grinding machines, then all machines must be down for one month during the six month period. So for example one vertical drill could be down in January and the other in April or they could both be down at the same time in March etc. For the grinding machines it is only necessary for two of the four machines to be down during the whole six month period.

### Assignment 7.1

Extend the previous model so it is up to the program to decide when the maintenance of each machine should take place rather than being pre-decided. You should still maximize profit as before. What is the value of the extra flexibility of allowing machine downtime to be chosen?

*Optimal objective value for Factory Planning Problem: £108855 (ultimo storage)*

*Optimal objective value for Factory Planning Problem: £109030 (primo storage)*

---

<sup>4</sup>Taken from Williams: “Model Building in Mathematical Programming”

## 6.8 Student Teacher Meetings

In Danish schools, it is customary to have meetings during the school year, often twice a year, between the student and parents and some of the teachers of the student. The meetings take place in the evening. Before the student-teacher meeting evening, the student (or parents!) make a list of the teachers they would like to meet with. When all students have submitted their teacher meeting request, a schedule needs to be made.

The teachers each use their own classroom for their meetings. The students and their parents walk around the school and meet with the teachers they wish to talk to. A number of time slots are given, e.g., a 10 minute interval, and your job is simply to plan when, i.e. in which time slot, the meetings will occur.

You can find data for the assignment on the book web-site <https://www.man.dtu.dk/MathProgrammingWithJulia>. Six different problem instances are available, where `StudentTeacherMeetingData_15_6.jl` have 15 students and 6 teachers:

```
StudentTeacherMeetingData_15_6.jl
StudentTeacherMeetingData_20_6.jl
StudentTeacherMeetingData_25_10.jl
StudentTeacherMeetingData_25_8.jl
StudentTeacherMeetingData_30_10.jl
StudentTeacherMeetingData_40_10.jl
```

In each file, the following information is given:

- $StudentTeacherMeetings_{s,t}$ : An incidence matrix, with one row for each student  $s$  and one column for each teacher  $t$ . If student  $s$  should meet teacher  $t$ , then  $StudentTeacherMeetings_{s,t} = 1$ , otherwise  $StudentTeacherMeetings_{s,t} = 0$
- $S$ : Number of students
- $T$ : Number of teachers
- $TS$ : Number of time slots

The meetings are private, so a teacher can only meet one family at a time. The families would all like to go home as early as possible from the meetings. Therefore, the overall objective is to minimize the sum of the latest time slot indices assigned to the families.

The problem is surprisingly difficult, so the data sets are all rather small and open-source solvers all struggle on even the smallest instances. This problem thus shows the value

of the expensive commercial solvers CPLEX and Gurobi.

As described in Section 3.7, it is possible to set a maximal solve time using

```
set_time_limit_sec(model, 600) # timelimit of 600 sec (5 minutes).
```

Note that the solver will not stop immediately after 600 seconds sharp, but wait to finish the current iteration of Simplex or what else it is doing. With a time limit on the solver, you will likely not manage to find the optimal solution, but you can check the value of the best solution that has been found (if any) using the usual `objective_value(model)`. You can then compare this with the dual bound (lower bound for minimization) at the time the solver stopped: `objective_bound(model)`. The optimal solution will have a value somewhere between these two.

### Assignment 8.1

Formulate the student-teacher meeting planning problem as a MIP, minimizing the sum of last time slots for each family. Implement your model in you Julia/JuMP.

*Hint:* You will need to keep track of the last time slot assigned to the various families, since this is what you are trying to minimize. For this you can exploit that it will be equal to or later than *any* time slot they have been assigned.

Only the optimal objective values for the first five problems are reported (together with the Gurobi solution time). These results were obtained using a Mac Book Pro, 2018.

*Optimal objective values for the student-teacher meeting problems*

- StudentTeacherMeetingData\_15\_6.jl : 66 (sec. 52.9)
- StudentTeacherMeetingData\_20\_6.jl : 110 (sec. 632.6)
- StudentTeacherMeetingData\_25\_8.jl : 131 (sec. 1393.3)
- StudentTeacherMeetingData\_25\_10.jl : 113 (sec. 1345.4)
- StudentTeacherMeetingData\_30\_10.jl : 152 (sec. 603.7)
- StudentTeacherMeetingData\_40\_10.jl : 248 ( $\geq 244$ , 1.61% gap) (sec. 929.7)

Here

## 6.9 Workplan for Teaching Assistants

A Mathematical Programming crash course, over 9 days, is taught by 4 teaching assistants (TAs) and 2 professors. The professors give the lectures and the TAs are available for help with the exercises. Suppose that four TAs (AL, FR, JE, and MI) are the TAs in the course. In this assignment, you have to plan when they should work during the course. Each TA must work exactly 52 hours (a total of 208 hours are available for the four TAs). The professors have attempted to come up with a plan of how many TAs to assign for exercise help in each of the 8 time slots from 9 to 17. This TA demand is given below in Table 6.4.

|       | Day |   |   |   |   |   |   |   |   |
|-------|-----|---|---|---|---|---|---|---|---|
|       | 1   | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
| 9-10  | 4   | 4 | 4 | 0 | 3 | 0 | 3 | 0 | 2 |
| 10-11 | 4   | 4 | 4 | 2 | 4 | 2 | 4 | 2 | 2 |
| 11-12 | 4   | 4 | 4 | 4 | 4 | 4 | 4 | 4 | 2 |
| 12-13 | 4   | 3 | 3 | 3 | 3 | 4 | 3 | 3 | 1 |
| 13-14 | 4   | 3 | 3 | 3 | 3 | 4 | 3 | 3 | 1 |
| 14-15 | 4   | 3 | 3 | 3 | 3 | 4 | 3 | 3 | 1 |
| 15-16 | 3   | 3 | 2 | 3 | 3 | 4 | 3 | 3 | 1 |
| 16-17 | 3   | 3 | 2 | 2 | 2 | 3 | 2 | 2 | 1 |

Table 6.4: TA demand for the first 9 days

To make the work easier for the TAs, we wish to take into account their personal preferences. Each TA has specified inconvenience scores for each day  $d$  and time period  $t$  in the matrix of inconvenience scores  $Inconvenience_{t,d}^{ta}$ . You can find these data on the book web-site <https://www.man.dtu.dk/MathProgrammingWithJulia>. You can then either simply copy-paste the data into your program or you can use the `include(<file name>)` command. Notice that the data also contains a simple normalization of the Inconvenience matrices.

### Assignment 9.1

Formulate a mathematical model that can be used to find the optimal work plan for the TAs which covers the demand but minimizes their inconvenience. Implement

your model using Julia/JuMP to find the optimal solution.

*Optimal solution TA work plan (minimal inconvenience): 94.98*

When you inspect the generated plan, you observe that the TAs work several disconnected times on the same day. This is rather inconvenient. You decide to modify your model and include the restrictions that each TA can at most work for one block of consecutive hours each day, and if they work on a day, they should have at least 2 hours of work that day.

### Assignment 9.2

Revise the mathematical model with the additional restrictions. Implement this model using Julia/JuMP to obtain the optimal plan.

*Optimal solution TA workplan (minimal inconvenience): 121.97*

Notice that while these extra requirements are very reasonable, it leads to a worse solution, regarding the TA inconvenience. On the other hand, the revised solution is an improvement for the TAs in a different way.

## 6.10 Scrap Removal

After cleaning up in the shed (Project scheduling), your father has identified a number of scrap items, which he needs to get rid of. There are 20 items of different weight as shown in Table 6.5. Your father contracts a haulage contractor to come and pick up big-bags of scrap. The contractor is paid €50 for each big-bag he has to transport away. Each big-bag can carry at most 100 kg of scrap and your father has ordered 10 big-bags. He makes the following deal with you: If you pack the items in big-bags, not breaking the 100 kg limit, and are able to use fewer big-bags, then **you** will get the €50 for each big-bag saved. How much money can you make?

|             | Items |    |    |    |    |    |    |    |    |    |
|-------------|-------|----|----|----|----|----|----|----|----|----|
|             | 1     | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 |
| weight (kg) | 35    | 10 | 45 | 53 | 37 | 22 | 26 | 38 | 63 | 17 |

|             | Items |    |    |    |    |    |    |    |    |    |
|-------------|-------|----|----|----|----|----|----|----|----|----|
|             | 11    | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
| weight (kg) | 44    | 54 | 62 | 42 | 39 | 51 | 24 | 52 | 46 | 29 |

Table 6.5: Weight for each of the 20 items

### Assignment 10.1

Formulate the problem of packing the scrap items in as few big-bags as possible without violating the weight limit as a MIP model. The goal should be to minimize the payment for the contractor. Solve the model using Julia/JuMP.

*Optimal objective for Scrap Removal: €400*

## 6.11 Groceries

The local grocer in your neighbourhood decides to offer delivery of goods to home and offers you the job of driving around with the deliveries. The job is simple: You go to the grocery shop 16.00, get the list of persons to deliver to and the bags of groceries in. You then make a round-trip in the grocery van, visiting all the customers, delivering their goods. When you return the van to the grocery shop, your work is over. You are paid a fixed amount each day, so the faster you can deliver the goods, the faster your job is over.

Since you want to work as little as possible, you want to plan the fastest possible route. On a given day there are 5 customers who ordered delivery and their coordinates are given below in Table 6.6. The grocery store is added as the first customer and is located in Origo (0,0). We will for simplicity assume that direct travel between the different locations is possible, so we can simply use euclidean distance between customers. Additionally distance and time are assumed directly related, so shorter is always faster. How can you then formulate a Mixed Integer Programming model to minimize the total round-trip tour length?

| Customer      | X coordinate | Y coordinate |
|---------------|--------------|--------------|
| Grocery store | 0            | 0            |
| 1             | 104          | 19           |
| 2             | 370          | 305          |
| 3             | 651          | 221          |
| 4             | 112          | 121          |
| 5             | 134          | 515          |

Table 6.6: Grocery and customer coordinates

You quickly realize that any round trip will consist of a number of different decisions, i.e. whether to travel from one customer to another or not. Each such decision can be defined using a binary decision variable  $x_{i,j} \in \{0,1\}$ , which is 1 if you travel from customer  $i$  to customer  $j$ . This makes it simple to calculate the total traveled distance that you wish to minimize as the objective:  $\sum_i \sum_j Dist_{i,j} \cdot x_{i,j}$ . The distances between customers,  $Dist_{i,j}$ , can advantageously be pre-calculated based on the coordinates given.

However, how can you define constraints so that only solution values of  $x_{i,j}$  which represents correct round trips are feasible? First you realize that any round-trip tour, means that you arrive and leave exactly **once** from each customer/store, i.e. looking at customer  $j$  there will be exactly one other customer/store,  $i$ , from which the van came when



arriving at  $j$  and so for exactly one  $i$  we have  $x_{i,j} = 1$ . Similarly for departing there will be only one customer to which the van goes when leaving  $j$ . These two requirements can be implemented with two abstract constraints. It is also necessary to fix the variables  $x_{i,i}$  to zero, i.e. going from customer  $i$  to customer  $i$  itself does not really make sense, so the value of these variables need to be fixed to zero.

### Assignment 11.1

Formulate the first model with the round trip distance to be minimized and the two different abstract constraints, ensuring that you travel to each of the customers and leave each of the customers.

*Optimal objective value for the grocery problem: €1576.85*

When you print out the values of the binary  $x_{i,j}$  variables, you do, however quickly realize that this is not a feasible solution, since it consists of two separate round-trips. But the solution **is** legal according to your model. Therefore, you need one or more additional constraints, which will make separate round-trips impossible. How to do that?

One of your friends suggests the following: Assign a counter (a decision variable)  $u_i \geq 0$  to each customer. This can simply be a positive continuous variable. Then create a constraint, which ensures that whenever you go from one customer to the next, the value of the counter is at least one unit larger than the customer you come from. At first this seems easy:

$$x_{i,j} + u_i \leq u_j \quad \forall i, j$$

You see e.g. that if customer  $i = 3$  was the first customer on the route and customer  $j = 5$  was the next, then with  $u_3 = 1$  the constraint gives  $u_5 \geq 1 + 1 = 2$  Which gives rise to a sensible counting. However, it also introduces a lot of relations between customer  $i$  and customer  $j$  when there should be no relation because  $x_{i,j} = 0$ . Going back to the former example and considering instead  $i = 5$ ,  $j = 3$  for which we know  $x_{5,3} = 0$  since the customers are in fact visited in the opposite order, the counting constraint would dictate  $0 + u_5 \leq u_3$  which contradicts the previous relation. To fix this, you use your knowledge from this course about turning constraint on and off and construct a bigM-type constraint with the bigM chosen as  $C$  defined as the number of customers (and the grocery store). Then re-write the constraint to:

$$u_i + 1 \leq u_j + C \cdot (1 - x_{i,j}) \quad \forall i, j$$

Notice, that if  $x_{i,j} = 1$ , we get exactly the previous formula, but if  $x_{i,j} = 0$  we add a large number on the right hand side, to allow any value of  $u_i$ .

Unfortunately, if you simply add this constraint as stated here, no solutions will exist. This is because we are working with a cycle where we have not yet defined any beginning or end. If the constraint is defined for all  $i, j$  as stated above, it would mean that since we start at the grocery store then  $u_1$  must be smaller than all the actual customers who are visited at some point later. However, as we return to the depot in the end as well,  $u_1$  also has to be greater than  $u_i$  for whatever customer,  $i$ , was visited last contradicting with  $u_1$  being the smallest  $u$ -value of all. The cycle of pairwise relations has to be broken somewhere, such as at the depot, by excluding that customer as a  $j$ -value from which the constraint applies.

### Assignment 11.2

Formulate the second model where you extend the previous model with counting variables and constraints which forces the counter variable to be one unit higher than the previous customer on the tour.

*Optimal objective value for the grocery problem: €1857.51*

After a one month trial period, the grocer opens up for taking more customers, and it is a great success, now there are suddenly 13 customers, coordinates given below in Table 6.7. It has now become a problem that the size of the grocery orders do not all fit in the van. It has been found that the van has a capacity of 26 units, and the size of the 13 customer orders are shown in Table 6.7.

| Customer  | X coordinate | Y coordinate | grocery size |
|-----------|--------------|--------------|--------------|
| 1 (Store) | 0            | 0            | 0            |
| 2         | 104          | 19           | 3            |
| 3         | 370          | 305          | 9            |
| 4         | 651          | 221          | 7            |
| 5         | 112          | 121          | 11           |
| 6         | 134          | 515          | 11           |
| 7         | 797          | 424          | 6            |
| 8         | 347          | 444          | 7            |
| 9         | 756          | 141          | 7            |
| 10        | 304          | 351          | 2            |
| 11        | 236          | 775          | 4            |
| 12        | 687          | 310          | 2            |
| 13        | 452          | 57           | 8            |

Table 6.7: Grocery and customers, coordinates, and delivery size

This presents a new problem: There is simply not room in the small van for all the groceries. Actually 3 vans of this size are required. So the grocer decides to acquire 2 more vans of the same size and hire 2 more delivery workers. But now you have a problem: Your nice model only works with **one** van.

The model needs to be updated to accommodate 3 vans. Since all the vans start going out of the grocery parking lot, the sum over the  $x_{1,j}$  variables, from the grocery to the customers, should be equal to 3. Similarly 3 vans should be going into the grocery store node. Instead of the counting variable counting units, we now count how much material has been delivered at a certain point on each route while simultaneously ensuring that the van capacity is respected. This can be done by changing the counting constraints to  $u_i + q_i \leq u_j + Q \cdot (1 - x_{i,j})$ , where  $Q$  is the capacity of the van, here 26.

**Note:** Solving this problem may take significant time using the open-source solver HiGHS. Consider adding a time limit.

### Assignment 11.3

Formulate the third model where you extend the previous model to handle 3 vans and take van capacity into consideration.

*Optimal objective value for the grocery problem: €5063.35. Solved on a fairly good laptop with HiGHS took 433 seconds (2026)*

## 6.12 Tennis

If you play tennis on a clay tennis court, then you must prepare it for the next couple of players. This is done by first brushing the clay and then sweeping the lines, using a special broom, see Figure 6.1.



Figure 6.1: Sweeping a tennis line

This assignment deals with the problem of finding the easiest (shortest) way of sweeping the lines (of one side) on the tennis court. In Figure 6.2, the outline of a tennis court is given, with all measurements in feet.

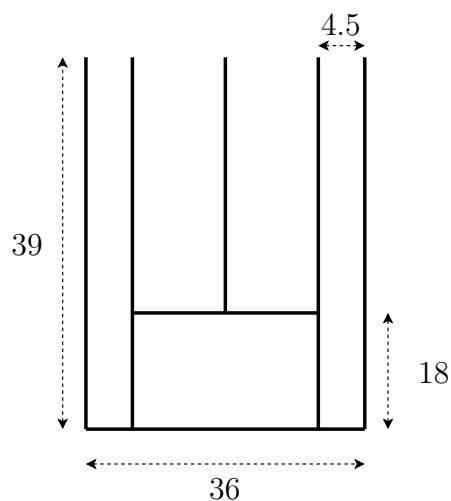


Figure 6.2: Court measurements (feet). The court is symmetric along the vertical axis.  
Note: Drawing not to scale.

The objective is to minimize the walking distance when sweeping the lines. When solving this problem, different formulations are possible.

*Hint:* Define a coordinate system and let all of the places where lines start or connect to other lines be points in the coordinate system. There are 12 such points. Then, calculate a distance matrix, which can easily be calculated in a couple of for-loops.

**Assignment 12.1**

If you are *only* allowed to step on the lines, then what is the best route you can walk, if you have to return to the starting position? This can be calculated analytically.

*Optimal distance: 390 feet*

**Assignment 12.2**

Construct a mathematical model to find the best route if you are allowed to walk between the lines, but still have to start and stop at the same points. Implement your model in Julia/JuMP.

*Optimal distance: 306 feet*

**Assignment 12.3**

Relax the model such that it is allowed to start and stop at any intersection, not necessarily the same. What is the best route now?

*Optimal distance: 280.5 feet*

**Assignment 12.4**

Constrain the model such that the start and stop places are only allowed on the outside lines (the 3 outside lines being the leftmost line, the bottommost line and the rightmost line in Figure 6.2). What is the best route?

*Optimal distance: 287.47 feet.*

## 6.13 Aircraft Landing

Aircraft movements at large airports are subject to a number of security constraints. You have been asked to schedule the landing sequence of ten aircraft at a busy European hub. Each aircraft has an earliest arrival time (corresponding to flying at its maximum speed) and a latest arrival time (mainly influenced by its fuel supplies). Within this window the aircraft has a target time, or the ideal arrival time of the aircraft. Due to the disruption it causes, any deviation from this target time (forwards or backwards) incurs a penalty per minute. Table 6.8 contains all relevant aircraft data.

| Aircraft          | 1   | 2   | 3   | 4   | 5   | 6   | 7   | 8   | 9   | 10  |
|-------------------|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| Earliest Arrival  | 125 | 187 | 94  | 99  | 111 | 123 | 129 | 131 | 142 | 155 |
| Target Time       | 155 | 212 | 134 | 127 | 123 | 138 | 145 | 157 | 162 | 179 |
| Latest Arrival    | 473 | 612 | 508 | 421 | 511 | 515 | 525 | 407 | 500 | 612 |
| Earliness Penalty | 10  | 20  | 35  | 25  | 35  | 30  | 30  | 50  | 30  | 35  |
| Lateness Penalty  | 10  | 20  | 35  | 25  | 35  | 30  | 30  | 50  | 30  | 35  |

Table 6.8: Aircraft data.

| Aircraft | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 |
|----------|----|----|----|----|----|----|----|----|----|----|
| 1        | -  | 5  | 15 | 15 | 15 | 15 | 15 | 10 | 15 | 5  |
| 2        | 5  | -  | 15 | 15 | 10 | 15 | 10 | 15 | 15 | 15 |
| 3        | 15 | 15 | -  | 10 | 15 | 10 | 15 | 10 | 15 | 10 |
| 4        | 15 | 15 | 10 | -  | 10 | 10 | 10 | 15 | 15 | 15 |
| 5        | 15 | 10 | 15 | 10 | -  | 10 | 5  | 10 | 15 | 5  |
| 6        | 15 | 15 | 10 | 10 | 10 | -  | 15 | 15 | 15 | 15 |
| 7        | 15 | 10 | 15 | 10 | 5  | 15 | -  | 10 | 5  | 10 |
| 8        | 10 | 15 | 10 | 15 | 10 | 15 | 10 | -  | 10 | 15 |
| 9        | 15 | 15 | 15 | 15 | 15 | 15 | 5  | 10 | -  | 10 |
| 10       | 5  | 15 | 10 | 15 | 5  | 15 | 10 | 15 | 10 | -  |

Table 6.9: Required aircraft separation time (minutes)

Due to turbulence and the time needed for an aircraft to clear the runway, a safety interval (or separation time) is needed between any two landings. Table 6.9 indicates the required separation times between two specific aircraft. This depends on, among other things, the sizes of the two aircraft. An entry in row  $p$  of column  $q$  states the minimum time needed between the landing of aircraft  $p$  and the subsequent landing of aircraft  $q$ , even if this is not consecutive.

**Assignment 13.1**

Formulate a mixed integer linear program that can be used to find the landing sequence that minimizes the total penalty incurred and satisfies the separation times. Each aircraft must land in the interval between its earliest and latest arrival (both included). Assume that landing times are required to be integer. Implement the model using Julia/JuMP to find an optimal solution.

*Optimal objective value: 2025*

**Assignment 13.2**

You have been asked to modify your solution to ensure that aircraft 5 and 6 do not land consecutively in either order. Modify your formulation above to ensure this and, using Julia/JuMP, find the best landing sequence satisfying the additional constraints.

*Optimal objective value: 2080*

**Assignment 13.3**

Consider the problem given in 13.2. You have now been asked to find the second best landing sequence. Modify your formulation to ensure that the second best landing sequence is obtained. Implement the model in Julia/JuMP and state the landing sequence (note that the second best sequence may have the same objective value as the optimal one).

*Optimal objective value: 2080*

## 6.14 Tariff Rates<sup>6</sup>

A number of power stations are committed to meeting the electricity load demands given in Table 6.10. The demand given refers to the whole time slot. As an example, 15000 MW of power are needed in the first time slot.

| Time slot   | Demand (MW) |
|-------------|-------------|
| 0.00 - 6.00 | 15000       |
| 6.00 - 9.00 | 30000       |
| 9.00 -15.00 | 25000       |
| 15.00-18.00 | 40000       |
| 18.00-24.00 | 27000       |

Table 6.10: Time slot demand, note that the time slots are not equally long.

There are 3 different types of electricity generators available. There are twelve of type 1, ten of type 2, and five of type 3. Each generator has to work between a minimum and a maximum level. There is an hourly cost of running each generator at the minimum level. In addition, there is an extra hourly cost for each megawatt at which a unit is operated above the minimum level. To start up a generator also incurs a fixed cost. All of this information is summarized in Table 6.11.

In addition to meeting the estimated load demands, there must be sufficient generators working at any time to make it possible to meet an increase in the load of up to 15%. This increase would have to be accomplished by adjusting the output of generators already operating within their permitted limits.

|        | Minimum<br>level | Maximum<br>level | Cost/hour<br>at minimum | Cost/(hour * MW<br>above minimum) | Start-up<br>cost |
|--------|------------------|------------------|-------------------------|-----------------------------------|------------------|
| Type 1 | 850              | 2000             | 1000                    | 2                                 | 2000             |
| Type 2 | 1250             | 1750             | 2600                    | 1.3                               | 1000             |
| Type 3 | 1500             | 4000             | 3000                    | 3                                 | 500              |

Table 6.11: Generator type specifications

---

<sup>6</sup>Taken from Williams: “Model Building in Mathematical Programming”



**Assignment 14.1**

Formulate a mathematical model to find the best daily electricity production plan that minimizes total cost. This plan will be used day after day (i.e. a *cyclic* production pattern where the time slot after the last one is the first the following day). Generators of the same type are indistinguishable so you may formulate your model to keep track of the total amount of each type of generator rather than the individual generators. Implement the model in Julia/JuMP and solve it.

*Optimal solution for Tariff Rates Problem: 988540*

**Assignment 14.2**

What would be the savings of lowering the 15% reserve output guarantee? In other words, what does this security of supply guarantee cost?

*Optimal solution for Tariff Rates Problem:  $988540 - 987790 = 750$*

*Tip 1:* Notice that you do not need to know *which* specific generator of each type is running, only *how many* of the different types are running.

*Tip 2:* If you get a solution with optimal value 1015150, make sure to include that the production pattern is *cyclic*.

## 6.15 Railway Depot Shunting

Passenger trains usually consist of multiple *units* coupled together. Units are categorized by unit types, where units of the same type have the same physical attributes (length, seat capacity, etc.). Over the course of a given planning horizon train units are coupled and decoupled from trains at certain depots to match the capacity of the trains in service to the expected passenger demand. Units not in service are parked (or stored) at depots, where they await their next service. This idle time at the depot can be used for maintenance purposes, among other things. Depots typically comprise multiple parallel tracks (which are not necessarily the same length) that can often only be accessed from one direction. In other words, the tracks operate as Last-In-First-Out (LIFO) stacks. We have tried to illustrate it (primitively) in the Figure 6.3. This figure shows a train with one units attached (top of the figure). This rail can drive to the right, past the railway shift point and then back up the unit into one of the 8 tracks. When collecting a unit, the train can obviously only take the unit last inserted into the track, hence the LIFO stack situation.

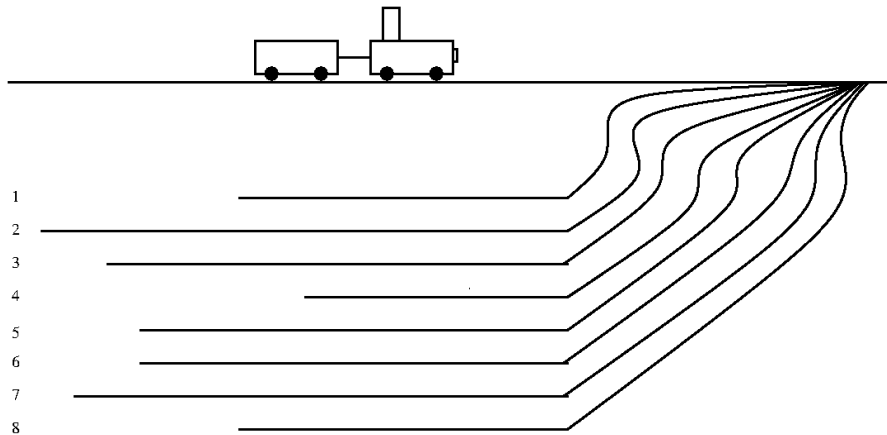


Figure 6.3: A railway shunting depot with 8 tracks.

You have been asked to consider one depot that contains eight LIFO tracks and investigate how many of the 70 train units assigned to the depot that can be feasibly parked there overnight. There are two types of units: Type One is 42m long, while Type Two is 84m long. Each train unit has a known arrival time in minutes after midnight (when it is taken out of service) and a known depot departure time (when it is needed back in service), again in minutes after midnight. This information has been made available in the data file `unit_arrival_departure_info.txt`. The respective track lengths are specified in Table 6.12.

Your aim is to determine how many of the 70 units that can be feasibly parked at the

|            | Track |     |     |     |     |     |     |     |
|------------|-------|-----|-----|-----|-----|-----|-----|-----|
|            | 1     | 2   | 3   | 4   | 5   | 6   | 7   | 8   |
| Length (m) | 500   | 800 | 700 | 400 | 650 | 650 | 750 | 500 |

Table 6.12: Track Lengths

depot without exceeding the track lengths and without "boxing in" units that need to depart earlier than another unit that has been parked in front of it. You may assume that a unit occupies depot capacity at its arrival time but has released depot capacity at its departure time, i.e. it is using the space during the time,  $t$ , for which

$$t_{Arrival} \leq t < t_{Departure}$$

(to simplify matters, the data in the data file `unit_arrival_departure_info.txt` are unique). Even though many units have the same type, their arrival and departure times are unique to the specific unit due to the maintenance requirement for the specific unit. This means that the departure and arrival times cannot be switched around even between similar units. Note that two units are in conflict if one blocks the departure of another. For some train units it will therefore never be possible to park them on the same track and this can be calculated solely based on departure and arrival times before writing up the model.

### Assignment 15.1

Formulate a mathematical model that can be used to determine the maximum number of units that can be feasibly parked in the depot. Implement your model in Julia/JuMP to determine what this value is.

*Optimal objective value: 63*

*Hint: Make sure to think about how you decide whether two train units can share a track or not. Additionally, you need to ensure that the track lengths are not exceeded, but you only need to check this every time a new train arrives since it is only at these times it could become exceeded.*

### Assignment 15.2

You have now been asked to investigate the impact of ensuring that only units of the same type can park on the same track. Modify your formulation to ensure this. What is the maximum number of units that can now be parked? Implement your model in Julia/JuMP to determine what this value is.

*Optimal objective value: 52*

## 6.16 Food Festival

A close friend of your mother is in charge of a food festival in your town, i.e. a number of stands in tents around the city, where food can be presented and tasted by the festival participants. There is, however, a planning problem at the festival: During the late evening, night and early morning, security guards are needed to watch the different facilities around the city. There is a list of 25 work shifts which needs to be assigned a security guard. However, some of the shifts cannot be covered by the same security guard because of overlap in time. Below in Table 6.13 is a list of the 25 shifts and for each shift the list of shifts which cannot be done by the same person. If a person is working shift 1 that person cannot work shift 2, shift 4, shift 6, shift 11, shift 17, shift 21, shift 22, and shift 25. Notice, that it goes both ways so a person working shift 25 cannot also work shift 1.

The concern is, how many persons is it necessary to hire for the shifts? It is easy to hire 25 persons and then let every person work exactly one shift. This will then not lead to any conflicts. Even though they will all receive the same hourly wage, there are a number of expenses associated with hiring the persons, therefore she wants to hire as few persons for the shifts as possible.

You can find the data for this problem on the book web-site

<https://www.man.dtu.dk/MathProgrammingWithJulia>. The data, which is also shown in Table 6.13, contains:

- $s \in Shifts = \{1, 2, 3, \dots, 23, 24, 25\}$
- $Conflict_{s1,s2}$ : An incidence matrix such that if  $Conflict_{s1,s2} = 1$  then shift  $s1$  and shift  $s2$  conflicts and cannot be worked by the same security worker.

### Assignment 16.1

Formulate a model which minimizes the number of persons it is needed to hire to cover all shifts without conflicts.

*Optimal objective value for the Food Festival problem: 5*

| Shift | Conflicting shifts |    |    |    |    |    |    |    |    |    |
|-------|--------------------|----|----|----|----|----|----|----|----|----|
| 1     | 2                  | 4  | 6  | 11 | 17 | 21 | 22 | 25 |    |    |
| 2     | 3                  | 7  | 9  | 14 | 16 | 17 | 23 |    |    |    |
| 3     | 4                  | 9  | 10 | 11 | 18 | 19 | 22 | 23 |    |    |
| 4     | 8                  | 9  | 10 | 19 | 20 | 21 | 24 | 25 |    |    |
| 5     | 6                  | 10 | 11 | 13 | 15 | 16 | 21 | 22 |    |    |
| 6     | 8                  | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 |
| 7     | 9                  | 10 | 14 | 15 | 21 | 22 | 25 |    |    |    |
| 8     | 9                  | 10 | 11 | 15 | 16 | 24 |    |    |    |    |
| 9     | 12                 | 13 | 15 | 19 | 21 | 24 |    |    |    |    |
| 10    | 11                 | 14 | 15 | 16 | 19 | 22 |    |    |    |    |
| 11    | 12                 | 13 | 15 | 19 | 20 | 21 | 25 |    |    |    |
| 12    | 14                 | 15 | 16 | 17 | 18 | 23 |    |    |    |    |
| 13    | 14                 | 16 | 19 | 23 | 25 |    |    |    |    |    |
| 14    | 15                 | 17 | 18 | 22 |    |    |    |    |    |    |
| 15    | 16                 | 17 | 23 | 24 | 25 |    |    |    |    |    |
| 16    | 18                 | 19 | 20 | 21 | 24 |    |    |    |    |    |
| 17    | 19                 | 22 | 24 |    |    |    |    |    |    |    |
| 18    | 20                 | 21 |    |    |    |    |    |    |    |    |
| 19    | 23                 | 24 | 25 |    |    |    |    |    |    |    |
| 20    | 22                 | 25 |    |    |    |    |    |    |    |    |
| 21    | 22                 | 23 | 24 |    |    |    |    |    |    |    |
| 22    | 24                 |    |    |    |    |    |    |    |    |    |
| 23    | 24                 | 25 |    |    |    |    |    |    |    |    |
| 24    | 25                 |    |    |    |    |    |    |    |    |    |

Table 6.13: List of shifts and conflicting shifts

## 6.17 The Wedding Planner

This assignment is about seating guests at tables, during a dinner. With data from a real wedding,  $G$  guests are to be seated at  $T$  tables. We want to find the best feasible seating-plan such that the guests at the same table share interests, have a somewhat equal distribution of men and women, and that each guest at least know some of the other guests at the table. We will assume that the tables are small, 9 seats, such that everybody at the table can talk to everybody. The feasibility requirements are:

- Each guest should sit at a table
- There is a limit of 9 people at each table
- Couples that attend the wedding should sit at the same table

You can find these data on the book web-site

<https://www.man.dtu.dk/MathProgrammingWithJulia>. There are two instances, the full data with 74 guests and a reduced instance with 20 guests. We will use the latter in this course due to how hard it becomes to solve the models for the full instance. You can then either simply copy-paste the data into your program or you can use the `include(<file name>)` command. The data contains the following parameters:

- Guests: The set of guests
- G: The number of guests
- Tables: The set of tables
- T: The number of tables
- SharedInterests: A two-dimensional matrix where `SharedInterest[g1,g2]` specify how many shared interests guest `g1` and `g2` has.
- Couple: An incidence matrix where `Couple[g1,g2]=1` if `g1` and `g2` are a couple
- Male: `Male[g]=1` if guest `g` is male
- Female: `Female[g]=1` if guest `g` is female
- Know: `Know[g1,g2]=1` if guest `g1` and guest `g2` know each other
- TableCap: Number of guests which can maximally be seated at the table, equal to 9.

The final model is rather complex, so we will iteratively build the model in four stages.

### Assignment 17.1

Formulate a mathematical model without an objective (so just a constant value) that can be used to solve this problem, i.e. find a seating that satisfy the constraints. Later we will add the various goals to the objective.

### Assignment 17.2

Add the following objective function to your model:  
*Maximize the number of shared interests for persons sitting at the same table*

*Optimal objective value: 67*

*Hint:* Define a new variable that takes the value of one if two guests are seated at the same table. If you get a higher result, then check to see if you are counting people twice or counting self-shared interests.

### Assignment 17.3

Now consider a different objective function: For each table, penalize the number of males and the number of females that are in excess of two. As an example, a table with seven males and two females has an excess of five males and incurs a penalty of three. Minimize the sum of penalties for all tables.

*Optimal objective value for: 0*

*Hint:* First formulate two sets of constraints that strictly enforce the requirements that there can be at most 2 too many of each gender. You can then introduce slack variables that allows these restrictions to be broken, but is penalized in the objective.



**Assignment 17.4**

Now consider a different objective function: You are to minimize the number of unfamiliar connections. The parameter *Know*, given in the datafile, defines which other guests each guest knows. The goal is that each person at the wedding should ideally sit at a table where that person knows at least 3 other people and if they do not know 3 at the table, the number less than three is the penalty. Thus a penalty of 1 for knowing only two persons but a penalty of 3 for knowing no one.

*Optimal objective value: 2*

*Hint:* Make a hard constraint that forces everybody to know at least 3 persons that they sit together with. You can then relax this constraint with a slack variable, which is minimized in the objective function.

When the bride & groom consider the seating plan, they really consider all three objectives *at the same time*. Hence, we have here a *multi-objective optimization problem*. To solve the combined problem we need to evaluate how important each of the objectives is. We consider them to be equally important.

**Assignment 17.5**

Solve your model again, but this time minimize the sum of:

- The negative number of shared interests (the negative of the objective in Assignment 17.2)
- The number of unbalanced male-female tables (the objective from Assignment 17.3)
- The number of insufficient connections (the objective from Assignment 17.4)

*Optimal solution for Wedding Planner Problem: -60*

The Wedding Planner problem is extremely hard. The actual wedding, where these data originates, had 74 guests. However, the model cannot be solved for this full dataset using even commercial general MIP solvers, but requires special algorithms.

## 6.18 Hot Air

This assignment is designed to give you an appreciation of the complexity of a very important real-life problem involving integer variables. It focuses on a simplified version of an airline fleet assignment problem.

*Hot Air* is an airline company that is considering an aggressive move into new territory in Texas. They have forecasts for the demand, i.e. the number of passengers that wish to travel on Hot Air, from one city to another city. Hot Air now needs to decide which routes to fly, that is, assign aircraft to. Their objective, not unexpectedly, is to maximize profit. Profit is generated by transporting fare-paying passengers, while taking into account any costs incurred when operating aircraft. All the necessary data are given below.

To make the problem manageable, Hot Air has made a number of simplifying assumptions, some more important than others. These are listed below.

### Simplifying assumptions:

1. No penalty is incurred for **not** transporting passengers that wish to travel; these passengers will just find another means of travel.
2. All passengers transported must be flown non-stop from their origin to their destination. No change of planes is allowed, nor is staying on a plane that travels via another city (*1-stop routes*). This assumption is relaxed in the last question.
3. Hot Air is seeking a daily schedule. They operate (like most airlines) the same schedule every day. A day is assumed to consist of 3 time periods, where planes can fly from any city to any other city: Dawn to mid-morning, mid-morning to afternoon, and afternoon to evening.
4. Due to restrictions on hangar lease and maintenance/service personnel (who work during the night), a specific number of planes must spend the night at each city. That number, of course, is the number of planes that depart each morning and return each evening.
5. Planes do not have to fly. A plane not flying does not incur any cost.
6. Planes are interchangeable: Hot Air essentially flies only one type of aircraft. Therefore, only the *number* of planes at any position at any time is important, not the identity of the planes.
7. More than one plane can fly a given route at the same time.

The data considers four Texas cities: Brownsville, Dallas, Austin, and El Paso. The aircraft type used by Hot Air is the Itty-Bitty-47, which holds a maximum of 120 passengers, and Hot Air has four of these planes. There is hangar capacity for two over-nighting planes in Brownsville and one each in Dallas and Austin. Three time periods per day are considered, each with its own demand for travel among the cities, see Table 6.14, Table 6.15 and Table 6.16. Ticket prices are given in Table 6.17. As an example a ticket from Austin to Brownsville costs \$109. The costs (per flight), which are the same across time periods, are given in Table 6.18. For instance, it costs \$4400 to fly from Brownsville to Austin. This includes landing fees, crew costs, fuel etc.

### Assignment 18.1

You have been hired to help Hot Air to optimize their daily schedule. Formulate a MIP model, implement it using Julia/JuMP, and present a solution to Hot Air management. During the flight to Hot Air's headquarters in the Windy City, you already realize that a possible set of decision variables would be  $y_{t,i,j}$ , which is the number of aircraft that fly during time period  $t$  from city  $i$  to city  $j$ , and  $x_{t,i,j}$ , the number of passengers being transported per time period  $t$  and from city  $i$  to city  $j$ . Of course, the latter is bounded by the number of people who want to fly, and the collective capacity of the planes flying on that route at that time.

*Optimal for Hot Air 1: 12411*

### Assignment 18.2

Hot Air liked your solution. Now, however, they are considering if profit could be increased by changing the hangar locations for the planes to stay overnight. There's a one-time cost for any possible change but you can disregard this, as it will not matter in the long run. Instead they ask you: If the hangar locations could be freely chosen, both in their locations and how many planes they can accommodate, then what would be the optimal daily schedule, where should the planes stay overnight and how much would it increase profits?

*Hint:* This is actually a simplification of the first model and can be solved quite elegantly.

*Optimal for Hot Air 2: 22368*

### Assignment 18.3

A manager at Hot Air wants to know about the actual profits for each flight and asks you to calculate these. Create a table that specifies the profit you make for each of the flights.

### Assignment 18.4

The manager is shocked to find out that Hot Air is actually *losing* money on certain flights and decides that for now, no flights are allowed if they generate a negative profit. Reformulate your model from Assignment 21.2 to include this requirement. What happens?

*Optimal for Hot Air 4: 20950*

### Assignment 18.5

Hot Air is beginning to feel the pressure from low-cost carriers and is looking for ways to compete with these. The marketing department discovers a new group of possible customers: Low-cost customers (e.g., students!) who can be attracted from bus transportation by lower prices and who do not expect premium service. The marketing department estimates that approximately an extra 50% of customers could be attracted if prices were lowered to 70% of the full-fair price. Hot Air does, however, *not* want to lower the price for the regular customers. Therefore, an employee from the marketing department comes up with the following idea: Only offer discount tickets at 70% of the price to customers who do *not* fly direct flights but have one stopover. Assume that you can still sell the full fair demand tickets, but also that you could additionally fill up the planes with discount passengers. Extend the model from the Assignment 21.2. Does this approach lead to higher profits?

**This part of the Hot Air assignment is optional and rather hard to solve. If you are ambitious, go ahead !**

*Optimal for Hot Air 5: 28355.6*

*Hint:* Remove the additional constraint from the previous question (engineers know better than managers) and consider introducing a variable to count passengers flying from city  $i$  to city  $j$  via city  $k$ . This refers to two flights, with the first flight in period  $t$  and the second in period  $t + 1$ . We will ignore discount travelers who wait for a period

or wait overnight. The new variable for the discount traveling passengers should then be added to the objective function *and* to the capacity constraints (they have to share planes with the normal passengers). When estimating the increased demand you should consider the time period of the *first* flight.

|             | Cities      |        |        |         |
|-------------|-------------|--------|--------|---------|
|             | Brownsville | Dallas | Austin | El Paso |
| Brownsville |             | 50     | 53     | 14      |
| Dallas      | 84          |        | 80     | 21      |
| Austin      | 17          | 58     |        | 40      |
| El Paso     | 31          | 79     | 34     |         |

Table 6.14: Demand, period 1

|             | Cities      |        |        |         |
|-------------|-------------|--------|--------|---------|
|             | Brownsville | Dallas | Austin | El Paso |
| Brownsville |             | 15     | 53     | 52      |
| Dallas      | 17          |        | 134    | 29      |
| Austin      | 24          | 128    |        | 99      |
| El Paso     | 23          | 15     | 30     |         |

Table 6.15: Demand, period 2

|             | Cities      |        |        |         |
|-------------|-------------|--------|--------|---------|
|             | Brownsville | Dallas | Austin | El Paso |
| Brownsville |             | 3      | 16     | 9       |
| Dallas      | 48          |        | 104    | 48      |
| Austin      | 62          | 92     |        | 68      |
| El Paso     | 13          | 15     | 21     |         |

Table 6.16: Demand, period 3

|             | Cities      |        |        |         |
|-------------|-------------|--------|--------|---------|
|             | Brownsville | Dallas | Austin | El Paso |
| Brownsville |             | 99     | 89     | 139     |
| Dallas      | 109         |        | 99     | 169     |
| Austin      | 109         | 104    |        | 129     |
| El Paso     | 159         | 149    | 119    |         |

Table 6.17: Ticket prices

|             | Cities      |        |        |         |
|-------------|-------------|--------|--------|---------|
|             | Brownsville | Dallas | Austin | El Paso |
| Brownsville |             | 5100   | 4400   | 8000    |
| Dallas      | 5100        |        | 11200  | 6900    |
| Austin      | 4400        | 11200  |        | 5700    |
| El Paso     | 8000        | 6900   | 5700   |         |

Table 6.18: Cost per takeoff

## 6.19 Wind Farm Cable Routing

A new offshore wind farm that comprises of 34 15MW turbines is being planned, and you have been asked to determine how best to connect the turbines to the substation (the location from which the generated power will be subsequently transformed and transferred to an onshore converter station). Turbines can be connected to another turbine or directly to the substation. A turbine can have any number of incoming connections, but is limited to at most one outgoing connection. The number of incoming connections (i.e., connected turbines) is limited by the capacity of the outgoing cable. There are three types of cables that can be used, and each differs in its capacity (i.e., the number of turbines it can support) and its cost per meter. The substation is limited to a maximum number of incoming connections. Your aim is to determine how to connect the turbines via each other to the substation and what cable types to use for the various connections such that the overall cost of the cable routing is minimized. Figure 6.4 gives an overview of the planned wind farm, complete with turbine locations (points 1-34) and the substation (point 35).

You have been asked to consider two **scenarios**. In the first scenario, the substation can have at most 4 incoming connections, and the cable types are as specified in Table 6.19. In the second scenario, the substation can have at most 3 incoming connections, and the cable types are as specified in Table 6.20. The distance matrix specifying the distance (in meters) between the locations of each pair of turbines is given in the file `wind_farm_data.txt`. This file also includes the coordinates of each turbine. In this version of the problem, we will assume that there is zero power loss in the cables.

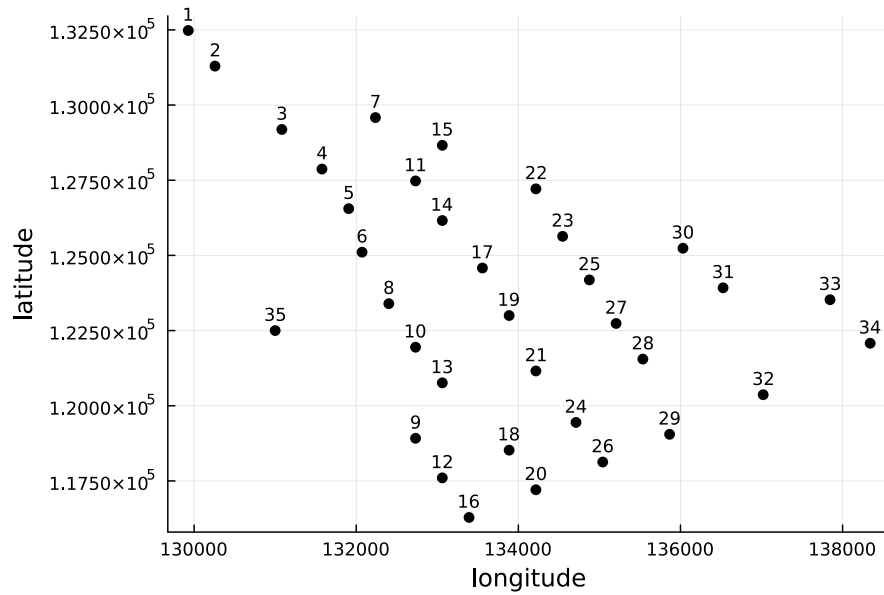


Figure 6.4: Wind farm overview

|                               | Cable Type |     |     |
|-------------------------------|------------|-----|-----|
|                               | 1          | 2   | 3   |
| Cost (€/m)                    | 135        | 250 | 480 |
| Max cable capacity (turbines) | 2          | 6   | 9   |

Table 6.19: Cable Type Data - Scenario 1 (up to 4 incoming connections to substation)

|                               | Cable Type |     |     |
|-------------------------------|------------|-----|-----|
|                               | 1          | 2   | 3   |
| Cost (€/m)                    | 170        | 290 | 610 |
| Max cable capacity (turbines) | 3          | 7   | 13  |

Table 6.20: Cable Type Data - Scenario 2 (up to 3 incoming connections to substation)

### Assignment 19.1

Formulate a mathematical model that can be used to find an optimal cable routing for the planned wind farm. Implement your model using Julia/JuMP to find an optimal solution to each scenario.



*Optimal objective value (Scenario 1): €12,058,455.40 (Gurobi: 18s, HiGHS: 203s)*

*Optimal objective value (Scenario 2): €12,791,103.40 (Gurobi: 25s, HiGHS: 260s)*

*Note:* The default value for the optimality gap of HiGHS is 0.01% so you may experience that your objective value is up to 0.01% larger than the above, yet the solver classifies it as optimal.

### Assignment 19.2

From an operational perspective, it is better if each "root branch" of the wind farm has more-or-less the same number of turbines, i.e., the routing is so-called *balanced*. A "root branch" refers to a connection from a turbine into the substation, and being balanced means that the number of supported turbines on all the connections going into the substation are similar. As an example if there are 3 connections into the substation we would prefer if they had 3, 2, 3 supported generators respectively, rather than 1, 2, 5.

Modify your formulation to ensure that the difference in the number of turbines between the root branch with the most turbines and the root branch with the least turbines is at most *one*. Implement your model using Julia/JuMP to find an optimal solution to each scenario.

*Optimal objective value (Scenario 1): €12,147,938.10 (Gurobi: 8s, HiGHS: 188s)*

*Optimal objective value (Scenario 2): €12,795,645.80 (Gurobi: 17s, HiGHS: 114s)*

*Hint:* If you see that your model becomes infeasible then double-check your new constraints. Have you ensured that you only limit the number of supported turbines on connections that exist?

|                               | Cable Type |     |     |
|-------------------------------|------------|-----|-----|
|                               | 1          | 2   | 3   |
| Cost (€/m)                    | 135        | 290 | 610 |
| Max cable capacity (turbines) | 4          | 6   | 12  |

Table 6.21: Cable Type Data - Scenario 3 (up to 3 incoming connections to substation)

**Assignment 19.3**

It is highly undesirable that cables between turbines cross each other. Using the data given in Table 6.21 and assuming that the substation can have at most three incoming connections, verify that in an optimal solution the cables between certain turbines do in fact cross. Identify, a constraint that can be added to the model to ensure that cables do not cross each other. Implement your model using Julia/JuMP to find an optimal solution in which cables do not cross. Remove the restriction that root branches should be balanced for this assignment. This is a **difficult** assignment, to reduce complexity see the hint below.

*Optimal objective value (crossings): €10,574,236.65 (Gurobi: 6s, HiGHS: 67s)*

*Optimal objective value (no crossings): €10,581,449.70 (Gurobi: 4s, HiGHS: 37s)*

*Hint:* Within large scale OR if you have a type of constraint of which there are very many but most of them will not be relevant, then instead of adding them all at once, making the model bigger than necessary, it can sometimes be beneficial to add them iteratively: Solve your model, identify constraints that are violated, add them and resolve until none are violated. This is especially common practice for sub tour elimination constraints in routing problems and you may want to consider doing something similar here.

## 6.20 Wedding Planner with Math-Heuristics

This assignment is a training exercise for using the two math-heuristics: Hamming-distance-heuristic and Fix-and-Optimize heuristic. Download the working code for both algorithms already implemented for the Wedding Planner problem, and the large dataset `WeddingData74.jl` (the actual wedding plan, with 74 guests). In order to make the problem into a good example, there are two changes to the Wedding Planner problem:

- The objective is (only) to maximize the shared interests. This simplifies the model and problem significantly.
- We have added a slack variable to allow that some guests are not seated. This however leads to a large penalty in the objective such that any good solution will have all guests seated.

Note that the provided code for the Fix and Optimize heuristic uses the Julia Package `Random` to randomly draw which variables to unfix. You may get an error in the line `using Random` if you do not have the package installed. To install simply use `using Pkg; Pkg.add("Random")` before running the code the first time.

### Assignment 20.1

Optimize (maximize) the Wedding Planner shared interest problem with the Hamming distance heuristic provided. Set a time limit for the overall algorithm of 2 minutes (120 seconds). Try a few different values for  $K_{initial}$  and see that you understand how the provided code changes  $K$  depending on whether the objective improves.

*Good solutions will have an objective value of 280 or higher.*

*Notice:* If using HiGHS as solver you may see that higher values of  $K$  reach the time limit of 120 seconds before even finding a feasible solution without slack. You can try to reduce  $K$ , increase the time limit or play around with the time limit provided to the MIP solver in each iteration of the Hamming Distance heuristic.

### Assignment 20.2

Optimize (maximize) the Wedding Planner shared interest problem with the Fix-and-Optimize heuristic provided for two minutes. Try different neighborhood sizes but note that the size of  $K$  for this heuristics does not change. You will want to have larger values of  $K$  compared to before, think about why that is and what is the maximum size  $K$  could take.

*Good solutions will have an objective value of 280 or higher.*

## 6.21 Math-heuristics for Stamps

It turns out that you uncle Archibald did not just leave you 100 stamps, but 400 and the auction got 2000 bids. Solving the Stamps problem with this much data directly using HiGHS or Gurobi is not possible (feel free to try).

In this exercise you are to implement two math-heuristics, Hamming-distance heuristics and Fix-and-optimize heuristics. You can use the math-heuristics from the Heuristic Wedding Planner problem. Replace the model and try to get good solutions. The larger dataset (bid\_data\_400\_2000.jl) is available on DTU-learn.

You can then either simply copy-paste the data into your program or you can use the `include(<file name>)` command.

### Assignment 21.1

Optimize (maximize) the Stamp problem with the Hamming-distance heuristic. Which  $K_{initial}$  neighborhood size achieves the best result in two minutes (120 sec.)?

*Good solutions will have an objective of at least 100.000*

### Assignment 21.2

Optimize (maximize) the Stamp problem with the Fix-and-Optimize heuristic. Which  $K$  neighborhood size achieves the best result in two minutes (120 sec.)?

## 6.22 University Timetabling

Creating timetables for universities is a complicated planning task. How the timetables are planned differs a lot from university to university, but in this assignment we will use data from the second university timetabling competition from 2007. These data comes from the University of Udine, Italy. During the questions in this assignment, the model will be expanded, ending up with the full model. Notice: The models becomes rather hard and the HiGHS solver will struggle, try to add a time limit but keep in mind that this means you will likely get a worse objective value than optimal, or no solution at all. The final model can only be solved by the Gurobi solver **and** only for some of the problems.

The basic assignment is to assign lectures to rooms and time slots, such that all lectures are planned. The basic data for the assignment are:

Sets of entities:

- $C$ : Number of courses
- $L$ : Number of lecturers
- $R$ : Number of rooms
- $D$ : Number of days
- $P$ : Number of periods each day
- $CU$ : Number of curricula (e.g. study lines)

Data:

- $penalty\_missing\_seat$ : Penalty for each student missing a seat
- $penalty\_room\_change$ : Penalty for each room more than 1 assigned to a course
- $penalty\_less\_than\_minimum\_working\_days$ : Penalty for each teaching day less than  $Course\_MinLectureDays_c$  for a course
- $penalty\_solo\_cur\_courses$ : Penalty for each isolated course for each curricula
- $Course\_NoStudents_c$ : Number of students for course  $c$
- $Course\_NoLectures_c$ : Number of lectures for each course  $c$
- $Course\_Lecturer_c$ : Index of lecturer teaching course  $c$

- *Lecturer\_NoCourses<sub>l</sub>*: No of courses for lecturer  $l$
- *Rooms\_size<sub>r</sub>*: Size of room  $r$  (how many students can it fit)
- *Course\_MinLectureDays<sub>c</sub>*: Minimum number of days required for teaching for course  $c$
- *Curricula\_CourseCurriculaIncidenceMatrix<sub>cu,c</sub>*: Incidence matrix, which is 1, if course  $c$  is part of curricula  $cu$

From DTU-learn, you can download 3 out of the 21 original dataset, the dataset *timetable1.jl*, *timetable4.jl* and *timetable5.jl*. These datasets can be directly included into the Julia program and contains the above data.

The basic feasibility requirement is that all lectures of all courses can be carried out. Notice, that it is accepted that there are not enough seats to the students. The requirements are:

- All lectures of all courses should be allocated
- At most one lecture can take place in a room for each timeslot on each day
- A lecturer can at most teach one course in each timeslot on each day
- Only one course for a curricula may be taught in each timeslot on each day
- For each course, there are a number of blocked timeslots, i.e. days and periods where the course cannot be taught by the lecturer. If *Blocked\_course\_timeslot* $[c, d, p] = 1$ , then course  $c$  cannot be taught on day  $d$  in period  $p$ . You can either formulate this as constraints, or use the fix command.

### Assignment 22.1

Formulate the feasibility model. If necessary declare the objective constant.

*Optimal objective value simply equal to what constant you have put in the objective function. See Table 6.22.*

| Data set      | Is optimal<br>solution found | Time used<br>in seconds | Best found<br>solution | Best lower<br>bound |
|---------------|------------------------------|-------------------------|------------------------|---------------------|
| timetable1.jl | Optimal                      | 0.7                     | 42.0                   | 42.0                |
| timetable4.jl | Optimal                      | 3.6                     | 42.0                   | 42.0                |
| timetable5.jl | Optimal                      | 1.9                     | 42.0                   | 42.0                |

Table 6.22: Table of result for Assignment 22.1. Solution time, using HiGHS on a Mac Book Pro (M2). Notice that when the found solution is optimal, then the best found solution objective and the lower bound is identical

Generally, our students like to sit down during the lectures! To avoid the risk of infeasibility we will allow to teach courses with more students than seats in the room, but penalize each student missing a seat for each lecture in each course. The size of the penalty factor *penalty\_missing\_seat* is given in the data.

### Assignment 22.2

Minimize the penalty of students missing seats.

*Optimal values enclosed in Table 6.23.*

| Data set      | Is optimal<br>solution found | Time used<br>in seconds | Best found<br>solution | Best lower<br>bound |
|---------------|------------------------------|-------------------------|------------------------|---------------------|
| timetable1.jl | Optimal                      | 3.4                     | 4.0                    | 4.0                 |
| timetable4.jl | Optimal                      | 6.8                     | 0.0                    | 0.0                 |
| timetable5.jl | Optimal                      | 5.4                     | 0.0                    | 0.0                 |

Table 6.23: Table of results for Assignment 22.2. Solution time, using HiGHS on a Mac Book Pro (M2). Notice that when the found solution is optimal, then the best found solution objective and the lower bound is identical

For both students and lecturers it is annoying if different lectures in a course is taught in different rooms. Hence the second objective is to minimize the number of used rooms for the courses. Naturally, each course will need to use at least one room. If **more** than one room is used, then the number of extra rooms are penalized, with the penalty *penalty\_room\_change*.

**Assignment 22.3**

Minimize the collective penalty for students missing seats **and** using more than one room for a course. Make sure to use the correct penalty weight for each penalty term.

*Optimal values enclosed in Table 6.24.*

| Data set      | Is optimal<br>solution found | Time used<br>in seconds | Best found<br>solution | Best lower<br>bound |
|---------------|------------------------------|-------------------------|------------------------|---------------------|
| timetable1.jl | Optimal                      | 35.1                    | 5.0                    | 5.0                 |
| timetable4.jl | Optimal                      | 176.5                   | 0.0                    | 0.0                 |
| timetable5.jl | Optimal                      | 40.9                    | 0.0                    | 0.0                 |

Table 6.24: Table of results for Assignment 22.3. Solution time, using HiGHS on a Mac Book Pro (M2). Notice that when the found solution is optimal, then the best found solution objective and the lower bound is identical

For some courses, the lecturers request that the lectures are spread out over time, e.g. if a course has 4 lectures, the lecturer may want those to be taught over at least 3 days. This request is reflected in the input data  $CourseMinLectureDays_c$ . Going back to the example, if all the lectures of the course is given the same day, the difference between the wished number of teaching days of the course and the actual number of teaching days, would then be  $3 - 1 = 2$ . Falling short of these wishes should then be penalizing in the objective with the penalty parameter  $penalty\_less\_than\_minimum\_working\_days$

**Assignment 22.4**

Add to you existing model such that you: Minimize the number of missing seats for each course, the number of extra rooms for each course **and** the penalty for using fewer days than requested for the courses.

*Optimal values enclosed in Table 6.25.*

Finally, students do not like to have spare time between teaching. This can be modeled in different ways, here we do the following: We want to avoid **isolated** curricula courses, i.e. courses for which no other courses for the same curricula are taught either before or after. For each isolated course for each curricula there is a penalty of  $penalty\_solo\_cur\_courses$ . **NOTICE:** With this extension the model becomes hard to solve, even for Gurobi so set a time limit of e.g. 5 minutes.



| Data set      | Is optimal<br>solution found | Time used<br>in seconds | Best found<br>solution | Best lower<br>bound |
|---------------|------------------------------|-------------------------|------------------------|---------------------|
| timetable1.jl | Optimal                      | 119.0                   | 5.0                    | 5.0                 |
| timetable4.jl | Optimal                      | 557.6                   | 0.0                    | 0.0                 |
| timetable5.jl | Optimal                      | 41.7                    | 15.0                   | 15.0                |

Table 6.25: Table of results for Assignment 22.4. Solution time, using HiGHS on a Mac Book Pro (M2). Notice that when the found solution is optimal, then the best found solution objective and the lower bound is identical

### Assignment 22.5

Minimize the number of missing seats for each course, the number of extra rooms for each course, the penalty for using fewer days than requested for the courses **and** the penalty for isolated curricula courses.

The addition of the last criteria in the objective function, makes the problem much harder. As can be seen in Table 6.26, HiGHS only finds **any** solution for the smallest dataset, timetable1.jl, and this solution is not optimal. For timetable4.jl and timetable5.jl, -1 is reported, corresponding to not even finding a feasible solution. Comparing to Table 6.27 Gurobi, finds optimal solutions to both timetable1.jl and timetable4.jl, but fails to find the optimal solution to timetable5.jl. Gurobi does, however, find a feasible solution, but this solution has a large gap between the best possible (bound) objective value (149) and the found solution (371).

| Data set      | Is optimal<br>solution found | Time used<br>in seconds | Best found<br>solution | Best lower<br>bound |
|---------------|------------------------------|-------------------------|------------------------|---------------------|
| timetable1.jl | Not optimal                  | 600.0                   | 17.0                   | 4.0                 |
| timetable4.jl | Not optimal                  | 600.1                   | -1.0                   | 7.1                 |
| timetable5.jl | Not optimal                  | 600.1                   | -1.0                   | 67.4                |

Table 6.26: Table of results for Assignment 22.5. Solution time, using HiGHS on a Mac Book Pro (M2). Notice that when the found solution is optimal, then the best found solution objective and the lower bound is identical

| Data set      | Is optimal<br>solution found | Time used<br>in seconds | Best found<br>solution | Best lower<br>bound |
|---------------|------------------------------|-------------------------|------------------------|---------------------|
| timetable1.jl | Optimal                      | 12.5                    | 5.0                    | 5.0                 |
| timetable4.jl | Optimal                      | 486.3                   | 35.0                   | 35.0                |
| timetable5.jl | Not optimal                  | 600.0                   | 371.0                  | 149.0               |

Table 6.27: Table of results for Assignment 22.5. Solution time, using Gurobi on a Mac Book Pro (M2). Notice that when the found solution is optimal, then the best found solution objective and the lower bound is identical

## 7 Multi-Objective Integer Programming

The Incredible Chairs company has a new challenge: Because of the general focus on climate all furniture is now rated according to it's CO2 emissions, on a scale from 1 to 10, where 10 is the best (lowest CO2 emissions) and 1 is the worst (highest CO2 emissions). How can the production be planned, taking into account **both** profit and climate rating ?

Recall the Incredible Chairs 3 MIP model, given  $c \in C$  chairs where production is limited by the production capacity of  $p \in P$  production lines. For each whole chair produced there is a profit of  $Profit_c$ . Each production line has a capacity limitation of  $Capacity_p$ , where each chair of type  $c$  produced requires  $RecourseUsage_{c,p}$  capacity of production line  $p$ . Our variables  $x_c \in Z^+$  specify how many of each chair should be produced. The MIP model for maximizing the Profit is given below:

$$\begin{array}{ll} \text{Maximize.} & \sum_c Profit_c \cdot x_c \\ \text{Subject to :} & \sum_c RecourseUsage_{c,p} \cdot x_c \leq Capacity_p \forall p \\ & x_c \in Z^+ \end{array}$$

The new CO2 rating is given below in Table 7.1.

|        | Chair types |   |   |   |   |   |   |   |   |   |
|--------|-------------|---|---|---|---|---|---|---|---|---|
|        | A           | B | C | D | E | F | G | H | I | J |
| Rating | 4           | 5 | 1 | 5 | 4 | 7 | 6 | 3 | 6 | 7 |

Table 7.1: CO2 Rating for each chair

To find a good production plan where both profit and rating is taken into account, a new model is formulated with **two** objective functions. See the model below.

$$\begin{array}{ll} \text{Maximize.} & \sum_c Profit_c \cdot x_c \\ & \sum_c Rating_c \cdot x_c \\ \text{Subject to :} & \sum_c RecourseUsage_{c,p} \cdot x_c \leq Capacity_p \forall p \\ & x_c \in Z^+ \end{array}$$

In the new model, there is no longer **one** unique best solution, but a whole **set** of solutions, in the Incredible chairs case the following solutions are all best, see the list below in Table 7.2.

| Solution no | Profit | Rating |
|-------------|--------|--------|
| Solution 1  | 18     | 12     |
| Solution 2  | 17     | 13     |
| Solution 3  | 16     | 14     |
| Solution 4  | 15     | 25     |
| Solution 5  | 13     | 27     |

Table 7.2: The 5 optimal solutions regarding profit and ranking

In Table 7.2 there are 5 solutions, which each are **Pareto optimal**. By this we informally mean that for each of these 5 solutions no other solution exists which are better (higher) in one of the objectives and at least as good on the other. As an example, a sixth solution with profit 16 and rating 13 would be uninteresting, as solution 3 has the same profit but a better rating.

Which of these 5 different production plans to choose is then up to the Decision Maker (DM), who must prioritize what is best for the company, pure profit, then solution 1 is best or if CO2 emissions are most important, then Solution 5. Hence, the optimal solution to a Multi-Objective Mixed Integer Programming model is a **set** of solutions, here 5.

Thanks to a very recent development, February 2023, it is now possible to formulate multi-objective mathematical programming models in Julia/JuMP. The model used to find the set of 5 solutions above for the multi-objective incredible chairs problem is given below.

### 7.0.1 Multi-Objective Incredible Chairs Julia/JuMP program:

---

```

1 #*****
2 # Multi-Objective Incredible Chairs 4
3 using JuMP
4 using HiGHS
5 using MultiObjectiveAlgorithms
6 #*****
7
8 #*****

```

---

```

9 # Data
10 Chairs=["A", "B", "C", "D", "E", "F", "G", "H", "I", "J"]
11 C=length(Chairs)
12 ProductionLines=[1, 2, 3, 4, 5]
13 P=length(ProductionLines)
14
15 Profit= [6, 5, 9, 5, 6, 3, 4, 7, 4, 3]
16 Rating = [4, 5, 1, 5, 4, 7, 6, 3, 6, 7]
17
18 Capacity=[47, 19, 36, 13, 46]
19 RecourseUsage=[
20 6 4 2 3 1 10 2 9 3 5;
21 5 6 1 1 7 2 9 1 8 6;
22 8 10 7 2 9 6 9 6 5 6;
23 8 4 8 10 5 4 1 5 3 5;
24 1 4 7 2 4 1 2 3 10 1]
25 *****
26
27 *****
28 # Model
29 MO_IC = Model()
30 @variable(MO_IC, x[1:C]>=0, Int)
31
32 # Objective 1: Maximize Profit
33 @expression(MO_IC, Profit_expr, sum(Profit[c] * x[c] for c=1:C))
34
35 # Objective 2: Maximize Rating
36 @expression(MO_IC, Rating_expr, sum(Rating[c] * x[c] for c=1:C))
37
38 # Define combined BI-OBJECTIVE
39 @objective(MO_IC, Max, [Profit_expr, Rating_expr])
40
41 # Capacity limitations
42 @constraint(MO_IC, [p=1:P], sum( RecourseUsage[p,c] * x[c] for c=1:C) <=
    ↪ Capacity[p])
43 *****
44
45 *****
46 # Solve
47
48 # Set MIP solver
49 set_optimizer(MO_IC, () -> MultiObjectiveAlgorithms.Optimizer(HiGHS.Optimizer))
50 set_silent(MO_IC)
51

```

```

52 # Set MO solver
53 set_attribute(MO_IC, MultiObjectiveAlgorithms.Algorithm(),
    ↪ MultiObjectiveAlgorithms.EpsilonConstraint())
54
55 optimize!(MO_IC)
56 #solution_summary(MO_IC)
57 #*****
58
59 #*****
60 # Solution
61 if result_count(MO_IC)>=1
62     println("No Pareto optimal points: ",result_count(MO_IC))
63     for i in 1:result_count(MO_IC)
64         y = objective_value(MO_IC; result = i)
65         println("Objective 1: ",round.(Int,y[1]), "      Objective 2:
    ↪      ",round.(Int,y[2]))
66     end
67
68 else
69     println("No solutions found")
70 end
71 #*****

```

---

We will now make a number of comments to this above Julia/JuMP program:

- The first part, line 1-6, imports the standard packages JuMP and HiGHS, but also a new one: MultiObjectiveAlgorithms. This package is necessary for the multi-objective part of the problem. You will need to install this package the same way as other packages. `using Pkg; Pkg.add("MultiObjectiveAlgorithms")`.
- The second data part, line 8-25, is similar to the previous Incredible Chair 3, only adding only one extra parameter: Rating.
- The model part, line 27-43, is slightly different:
  - Declaring the variables is as usual, line 30
  - Declaring the constraints is as usual, line 42.
  - Declaring the **objective function** is different. First an expression for each objective in line 33 and line 36. Finally, the two expressions are declared to be a vector objective function, line 42.
- The solve part, line 45-57, is significantly different. First the MultiObjectiveAlgorithms optimizer is told to use the HiGHS solver as a sub-routine in solving the

multi-objective model. Then one of the available bi-objective optimization algorithms offered in the `MultiObjectiveAlgorithms` is selected for this problem in line 53. With these definitions, the problem can be optimized.

- The solution part, line 59-71, is slightly different, since what is returned is not just one solution, but a whole list of solutions. The number is found using the `result_count` function.

## 7.1 The importance of Multi-Objective models

When solving real-world problems it quickly becomes clear that the real-world is multi-objective. When humans look at solutions to planning problems, they will most often evaluate the solutions according to a list of different measures (objectives).

- Project scheduling. Here there are two objectives: Minimize the used money and minimize the used time.
- Chair Logistics 2. Here there are two costs which are minimized, transport costs and depot costs. Even though both costs are measured in money, some of the costs are bound as fixed costs in the depots whereas other costs, the truck transportation costs, are variable.
- Tariff Rates. At first glance, only one objective is used, money. But the requirement that at least 15 % extra demand should be possible to be produced by the running plants. These 15 % could instead be considered a second objective, to be maximized.
- The Wedding Planner. Here there are 3 objectives, maximize the shared interests, minimize the gender imbalance at the different tables and minimize the number of people with no friends at the tables.
- Hot Air. Here the objective is money, but there are two different types of tickets: Regular and discount tickets. Both objectives can be maximized.

## 7.2 Multi-objective theory

Multi-objective optimization has been researched for at least the last 50 years. The state of both the theory and the solvers available for practitioners is unfortunately less mature as single objective optimization. Given the importance of multi-objectiveness in real-life problems, we decided to include this in the course. We would however like to stress that this chapter provides only a very brief introduction. We would also like to stress that the optimization algorithms available in Julia/JuMP may still be somewhat

unreliable.

In this course, we utilize a very recent addition to the JuMP package: In March 2023 the possibility to model with vector objective functions was introduced. To the best of our knowledge, this is the first modelling language where this is possible.

The general multi-objective Mathematical Program we consider in this course is given below. In this course, we only consider multi-objective Mathematical Program with linear objective functions and linear constraints (as illustrated below).

$$\begin{array}{ll}
 \text{Maximize/Minimize} & \begin{array}{l} \sum_i F_i^1 \cdot x_i \\ \sum_i F_i^2 \cdot x_i \\ \dots \\ \sum_i F_i^k \cdot x_i \end{array} \\
 \text{Subject to :} & \begin{array}{l} \sum_i A_{i,j} \cdot x_i \leq B_j \quad \forall j \\ x_i \in R, Z, \{0, 1\} \end{array}
 \end{array}$$

The model above, has  $k$  objective functions. We will assume that all the objective functions are either maximized or minimized. Notice, that to achieve minimization of an objective function is the same as maximizing the function multiplied by  $-1$ . A very important point is that there is usually not just one optimal solution but many.

To compare solutions we introduce the concept of *(Pareto) dominance*: A feasible solution  $\mathbf{x}$  *(Pareto) dominates* another solution  $\mathbf{y}$  if  $f_i(\mathbf{x}) \leq f_i(\mathbf{y}) \quad \forall i \in \{1, 2, \dots, k\}$  and  $f_j(\mathbf{x}) < f_j(\mathbf{y})$  for at least one  $j \in \{1, 2, \dots, k\}$  (assuming minimization). Thus, no other solution can dominate a Pareto optimal solution. We will use the terms *Pareto dominant solutions* or *non-dominated solutions* interchangeably. When solving a multi-objective optimization program like the one above, the goal is to find the set of all non-dominated solutions. This set we call the Pareto Front.

There are many different types of multi-objective mathematical programming. They depend on a number of parameters:

- The number of objectives
- Whether all variables are continuous. Then the model will be a Multi-Objective Linear Program (MOLP)
- Whether there are continuous variables in two or more objective functions
- If the objective functions only contain integer variables **and** all the objective function coefficients are integer ( $F_i^1 \in Z$ )



In this book we only consider the simplest versions of the multi-objective mathematical programming models. Hence we will restrict our attention to multi-objective optimization with two objective functions. Furthermore, we will only consider problems where the Pareto front only consists of points, which is illustrated below.

For multi-objective problems with only two objectives we can illustrate different types of Pareto fronts. If the multi-objective mathematical programming model only contains continuous variables, i.e. is a MOLP model, the Pareto Front becomes continuous, see Figure 7.1, where we assume maximization. In this case there is an **infinite number** of solutions on the Pareto Front, corresponding to the connected line Figure 7.1.

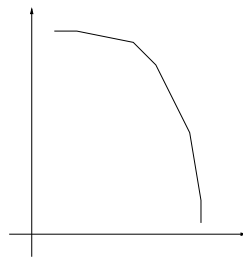


Figure 7.1: A continuous Pareto Front. The two axis correspond to the two objectives.

If a multi-objective **mixed integer** programming model, where one of the objective functions only contains integer/binary variables, then the Pareto Front becomes point-wise, see Figure 7.2, where we assume maximization. In Figure 7.2 the Pareto Front consists of a finite number of Pareto points, corresponding to the dots.

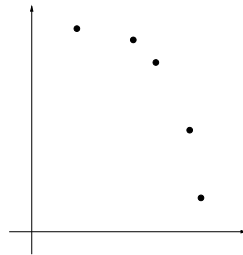


Figure 7.2: A point-wise Pareto Front

When performing multi-objective optimization it is much easier to consider point-wise Pareto Fronts, and we will in this course **only** consider point-wise Pareto front problems. A final simplification is sometime present in the models. Beside being point-wise, if only integer/binary variables are in the objective functions **and** all the coefficients in the objectives are integer, i.e.  $F_i^d \in \mathbb{Z}$ , then the objective functions can naturally only take integer values. In Figure 7.3 it is illustrated that all the point solutions are placed in a grid. Having only integer valued Pareto solutions often simplifies the work for the multi-objective optimization algorithms.

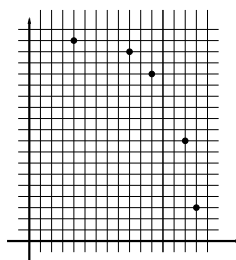


Figure 7.3: A point-wise Pareto Front in an integer grid

## 7.3 Multi-objective optimization algorithms

As mentioned earlier, access to multi-objective optimization through JuMP is a very new thing and the software is still under construction. Using JuMP, through the Julia package `MultiObjectiveAlgorithms`, 6 different algorithms are offered to the users.

The main one and the only one we will use in this course is the  $\epsilon$ -Constraint algorithm. In the code example in Example 7.0.1 it is setup in line 49 and line 53. The  $\epsilon$ -Constraint algorithm uses a MIP solver as a sub-routine, and which solver is used (described in more detail in the section below), is setup in line 49. This setup line is necessary for all the multi-objective algorithms available through JuMP. The second part is that actual choice of which multi-objective algorithm to use, is given in line 53. The alternatives to the  $\epsilon$ -Constraint algorithm are:

- Epsilon Constraint Chankong and Haimes (2008). This is the classic  $\epsilon$ -constraint method, invented in 1983.
- Dichotomy Aneja and Nair (1979). Notice that this method does **not** find the full pareto front, only the extreme points of the pareto front, i.e. the solutions which can be found using different secularization's of the objectives.
- Chalmet Chalmet et al. (1986)
- DominguezRios Domínguez-Ríos et al. (2021)
- KirlikSayin Kirlik and Sayin (2014)
- TambyVanderpooten Tamby and Vanderpooten (2021)

## 7.4 The epsilon constraint algorithm

The simplest multi-objective optimization algorithm to solve a two dimensional multi-objective problem is the **epsilon-constraint** ( $\epsilon$ -constraint) algorithm. The basic layout is given below.

**Algorithm 1:** The  $\epsilon$ -Constraint algorithm

```

1 solution-exists=true;
2 while solution-exists do
3   (solution-exists,z1,z2)=LexioGraphicZ2Z1();
4   if solution-exists then
5     AddParetoPoint( (z1,z2) );
6     SetLowerBoundZ1(z1+ $\delta$ );

```

In the pseudo-code above an  $\epsilon$ -Constraint algorithm for a bi-objective maximization problem is shown. First a boolean variable is set to true (line 1). This variable is used to determine if the algorithm should terminate. Then the main while loop is executed (line 2). A function, which performs lexicographic optimization, is called: First maximize regarding  $z_2$ , then add a lower bound on  $z_2$ , such that it cannot become worse than the solution just found, and optimize (maximize) regarding  $z_1$ . If no lexicographic solution is found the Lexicographic function should return the value **false** to the solution-exist variable, and the algorithm will terminate. If a solution with the current limitation exists, it is part of the front and is saved (line 5). Finally, the lower bound on  $z_1$  is increased, above the  $z_1$  value of the current solution (line 6). If the problem is a point wise problem in an integer grid, see Figure 7.3, then setting  $\delta > 0 \wedge \delta < 1$  will lead to an optimal Pareto front, given enough time.

We illustrate the  $\epsilon$ -Constraint algorithm graphically below in 4 steps in Figure 7.4 to Figure 7.7. In Figure 7.4 the leftmost Pareto point is found, called the Upper Left (UL) Pareto point. Denoting this point  $(z_1^*, z_2^*)$  then in Figure 7.5, a lower bound on  $z_1$  is added:  $z_1 \geq z_1^* + \epsilon$ , making UL infeasible. In Figure 7.6 the lexicographic optimization,  $z_2$  first then  $z_1$ , is used to find the next point. This continues until the Lower Left (LL) Pareto point is found. When it is found, we however do not know that it is found. Only, when another bounding constraint is added, making LL infeasible and lexicographic optimization,  $z_2$  first then  $z_1$ , is used but it returns infeasible, we know that the LL has been found, and the Pareto front is complete.

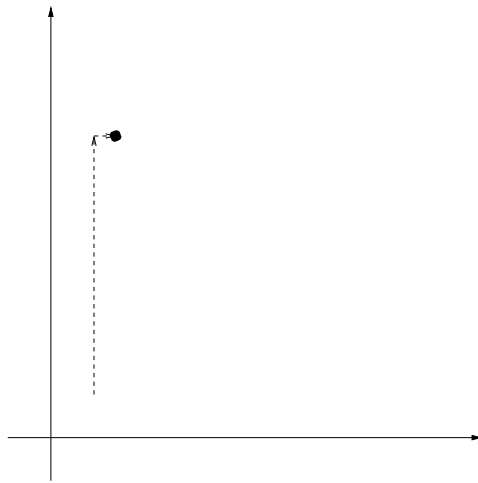


Figure 7.4: Finding Upper-Left (UL) Pareto point through lexicographic  $z_2 \rightarrow z_1$  optimization

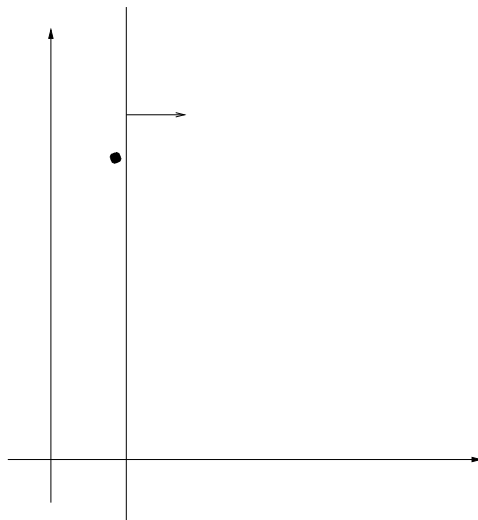
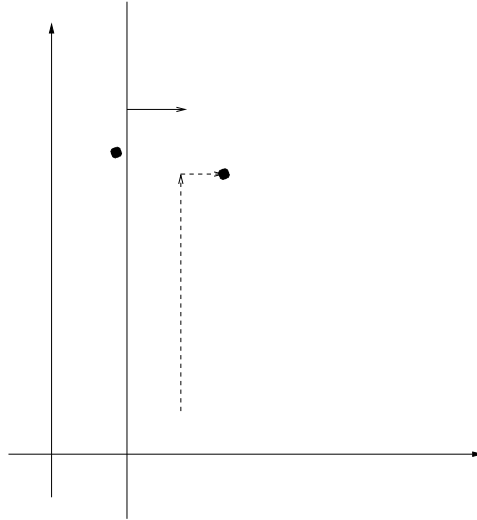
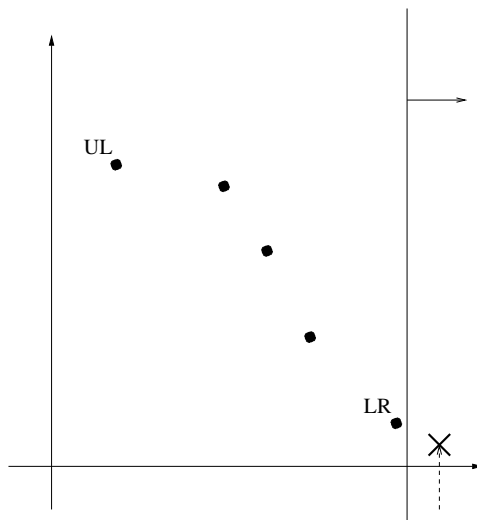


Figure 7.5: Adding bounding constraint on  $z_1$

Figure 7.6: Finding next Pareto point through lexicographic  $z_2 \rightarrow z_1$  optimizationFigure 7.7: Finding **no** solution after last point, Lower Left (LL).

A few implementation advices when implementing the  $\epsilon$ -Constraint algorithm in Julia/JuMP:

- In the Lexicographic function, it is necessary to change the objective function, to first maximize  $z_2$  and then maximize  $z_1$ . This is actually easy: Whenever a new "@objective" command is issued, the current objective function is replaced with the new version (unlike "@constraint" which are added to the existing model).
- Lower limiting the value of the  $z_1$  variable is also relatively simple: Either set the lower bound on the variable, or add a constraint. Notice that the previous

constraints are not removed, but have a lower right-hand-side value for  $z_1$ , and can hence be ignored, since e.g.  $z_1 \geq 1$  is always satisfied if you also restrict that  $z_1 \geq 2$ .

## 8 Multi-Objective optimization Programming problems

## 8.1 Multi-objective Startup Fund

The manager of the Startup Fund manager now hear about the 17 Sustainable Development Goals (SDG) and start to wonder how SDG friendly his investment is. Hence he hires an Non-Governmental Organsation (NGO) to evaluate the 9 investments according to the Sustainable Development Goals (SDG) and rate the different investment on a scale between 1 (worst) and 10 (best).

|     | Investment |   |   |   |   |   |   |   |   |
|-----|------------|---|---|---|---|---|---|---|---|
|     | 1          | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
| SDG | 8          | 6 | 8 | 3 | 4 | 5 | 3 | 2 | 7 |

Table 8.1: SDG rating from 1 to 10

You might wonder why the investment manager bothers! Now he may be personally invested in SDG, but really it is about **risk** avoidance. If he invests in a company which ruins the clean water in it's neighbourhood (not fulfilling SDG 6) there is a high probability that at some point there is going to be blow back either from government or from the local population. And this is probably bad for business ! There are 17 different goals (<https://sdgs.un.org/goals>) so it is very difficult to evaluate all of them, hence the overall rating according to all the goals. In this version of Startup fund Problem, we use the **first** version (Assignment 2.1) without the extra demands to the composition of the investment.

### Assignment 1.1

Find the investment portfolio that gives the best possible SDG value, ignoring profits.

*Optimal objective value for Startup Fund 2, SDG: 29*

### Assignment 1.2

Now the startup investment manager becomes worried since profit was ignored in the model but maybe the good SDG solution can be improved regarding the profit without reducing the SDG score. Hence he wants the solution that maximizes SDG first, and without allowing SDG to fall below the optimal value *then* optimizes profit. This is called the Lexicographic SDG-profit solution.



The Lexicographic SDG-profit solution, SDG: 29 and profit: 116. Notice, that this is actually the same result as in the previous assignment. But there we could not know that there would actually not be a better profit, with an SDG of 29. Now we know it to be this way.

### Assignment 1.3

Then look at the other extreme: The Lexicographic profit-SDG solution. (Optimise profit first and then find the best SDG without compromising profit)

The Lexicographic SDG-profit solution, profit: 191 and SDG: 19

To get the best data for making the investment decision, the startup fund manager now requests the **full** Pareto front, where the Lexicographic points are the two extreme points.

### Assignment 1.4

Find the entire Pareto front, regarding SDG objective and profit objective

*Optimal bi-objective Pareto Front*

|            |         | Investment |       |       |       |       |       |       |       |       |
|------------|---------|------------|-------|-------|-------|-------|-------|-------|-------|-------|
|            |         | 1          | 2     | 3     | 4     | 5     | 6     | 7     | 8     | 9     |
| Profit Obj | SDG Obj | $x_1$      | $x_2$ | $x_3$ | $x_4$ | $x_5$ | $x_6$ | $x_7$ | $x_8$ | $x_9$ |
| 116        | 29      | 1          | 1     | 1     | 0     | 0     | 0     | 0     | 0     | 1     |
| 122        | 28      | 1          | 0     | 1     | 0     | 0     | 1     | 0     | 0     | 1     |
| 134        | 27      | 1          | 0     | 1     | 0     | 1     | 0     | 0     | 0     | 1     |
| 141        | 26      | 1          | 1     | 1     | 0     | 1     | 0     | 0     | 0     | 0     |
| 149        | 25      | 1          | 0     | 1     | 0     | 0     | 0     | 0     | 1     | 1     |
| 156        | 24      | 1          | 1     | 1     | 0     | 0     | 0     | 0     | 1     | 0     |
| 162        | 23      | 1          | 0     | 1     | 0     | 0     | 1     | 0     | 1     | 0     |
| 174        | 22      | 1          | 0     | 1     | 0     | 1     | 0     | 0     | 1     | 0     |
| 178        | 21      | 1          | 0     | 0     | 0     | 1     | 0     | 0     | 1     | 1     |
| 191        | 19      | 1          | 0     | 0     | 0     | 1     | 1     | 0     | 1     | 0     |

Table 8.2: Profit-SDG Pareto front

## 8.2 Multi-objective Young Couple 2

Eve and Steven realize that their original work distribution problem is actually multi-objective (bi-objective): They want to minimize the total amount of time used on the work **and** they want to ensure that the time each of them spend is as similar as possible. Furthermore, they want to include additional chores, on top of the 4 jobs given in the first version of the assignment. The work time for the new jobs is given in Table 8.3:

|        | clean | cook | dishwashing | laundry | lawn | shopping | carwash |
|--------|-------|------|-------------|---------|------|----------|---------|
| Eve    | 4.5   | 7.8  | 3.6         | 2.9     | 1.1  | 2.1      | 1.9     |
| Steven | 4.9   | 7.2  | 4.3         | 3.1     | 2.3  | 1.5      | 3.2     |

Table 8.3: Task times (hours)

### Assignment 2.1

Convert the model to a bi-objective model, where both the total worktime and the difference in worktime (always positive), is minimized.

| Total work time | Work time difference | Eve worktime | Steven worktime |
|-----------------|----------------------|--------------|-----------------|
| 22.7            | 5.3                  | 14.0         | 8.7             |
| 22.9            | 0.7                  | 11.1         | 11.8            |
| 24.0            | 0.2                  | 12.1         | 11.9            |
| 27.0            | 0.0                  | 13.5         | 13.5            |

Table 8.4: List of 4 different Pareto optimal work-plan solutions

With the list of Pareto optimal solutions in Table 8.4 Eve and Steven can now ponder on a classical planning problem: **What is the value of equality?**

## 8.3 The Multi-objective Wedding Planner

The wedding planner had 3 objectives:

1. Maximize the sum of shared interests at each table
2. Minimize the difference in the number of men and women at each table above 2
3. Minimize the number of guests who do not know at least 3 at their table

In this assignment you should solve the problem bi-objectively with the following two objectives:

1. Maximize the sum of the shared interests
2. Maximize the sum of the **negative** number gender differences plus the **negative** number of guest who know less than 3 at their table

### Assignment 3.1

Find the entire Pareto front, for the bi-objective version of the Wedding Planner problem for the reduced dataset `WeddingData20.jl`.

*Optimal bi-objective Pareto Front:*

| Shared Interests       | Negative Gender + Know |
|------------------------|------------------------|
| <b>Objective 1: 48</b> | <b>Objective 2: -2</b> |
| Objective 1: 49        | Objective 2: -5        |
| Objective 1: 62        | Objective 2: -5        |
| Objective 1: 64        | Objective 2: -5        |
| <b>Objective 1: 65</b> | <b>Objective 2: -5</b> |
| Objective 1: 66        | Objective 2: -8        |
| <b>Objective 1: 67</b> | <b>Objective 2: -8</b> |

Table 8.5: Pareto front, found using `MultiObjectiveAlgorithms.EpsilonConstraint()`. Notice the algorithm returns 4 dominated solutions the ones that are not **bold**)

## 8.4 Multi-objective Aircraft Landing

Let us now consider a bi-objective variant of the Aircraft Landing Problem from Section 6.13. While it is extremely important to ensure that the aircraft arrival times deviate as little as possible from their respective target arrival times, it is also important to ensure that the time between consecutive aircraft landings is as large as possible (to prevent the propagation of any unforeseen delays). Changing the landing sequence is seen as undesirable as ground operations at the airport are made aware of, and anticipate, a certain landing sequence.

### Assignment 4.1

Modify your formulation from Assignment 13.2 (assume that aircraft 5 and 6 can still not land consecutively in any order), so that it now maximizes the minimum separation time between any aircraft landings, instead of minimizing deviation penalties. You may want to consider having a variable for the minimum separation time. Implement your model in Julia/JuMP to find an optimal solution to this *single-objective problem*.

*Optimal objective value: 48*

*Hint:* If your objective value is too *small*, e.g. 44, then consider whether you have a bigM constraint with a too small value so you end up restricting the value of the minimum separation time by accident.

### Assignment 4.2

You observe that with this formulation, the penalty associated with target time deviation can be arbitrarily large. By modifying your formulation, find the landing sequence with the total minimum penalty, subject to the requirement that the separation time must be at least 48 minutes between any two landings.

*Optimal objective value: 63410*

### Assignment 4.3

Now remove the constraint on minimum separation time and instead consider the bi-objective version of the problem in which you simultaneously try to minimize the total penalty incurred and maximize the minimum separation time between any two aircraft landings. Explore the trade-off between the two objective functions by implementing your model in Julia/JuMP to obtain the entire pareto. **Note: Gurobi is the suggested solver for this assignment. HiGHS will work but**

be prepared to wait some time.

*Number of pareto optimal solutions: 49*

*Should include:*

*Separation time = 48, Penalty = 63410 (as seen in Assignment 4.2)*

*Separation time = 0, Penalty = 2080 (seen in Assignment 13.2 )*

## 8.5 $\epsilon$ -Constraint algorithm

Implement the  $\epsilon$ -Constraint algorithm described in Section 7.4, and apply it to one of the previously implemented Multi-Objective problems, e.g. Startup Fund, Young Couple, Wedding Planning or Aircraft Landing.

## 9 Data

In Julia there are many different data types. The data types we need in this book for our LP models and MIP models are simple: Scalar numbers and arrays of numbers of different size and dimensionality.

### 9.1 Direct data definition

In the assignments, we need arrays of different dimensions:

- Scalar numbers (zero-dimensional arrays):

```
f=90
Q=31
```

- Vectors (one-dimensional):

```
customer_orders=[11 7 6 12]
C=length(customer_orders)
```

Notice: The space character separates the numbers. Here we have a vector with 4 numbers. This number can be found using the length function. To access the 3 element simply write:

```
customer_orders[3]
```

The index always starts at 1 and ends at the length of the array, here 4. If an attempt is done to access an element outside the array, e.g. 0, Julia will give an error.

- Matrix's (two-dimensional):

```
distances=[
17 39 84 ;
109 32 98 ;
```

```
139  4  31 ;
123 129 116 ]
(I,J)=size(distances)
```

Notice: Again the space character separates the numbers, but additionally the semicolon separates the different rows. The size function can be used to find that there are 4 rows (I=4) and 3 columns (J=3). To access the 3 element of the 4 row write:

```
distance[4,3]
```

which has the value 116. For each dimension index validity is checked.

- Matrix's (three-dimensional):

```
time_distances=zeros(Int16,2,4,3)
time_distances[1,:,:]=[
17  39  84 ;
109 32  98 ;
139  4  31 ;
123 129 116 ]
time_distances[2,:,:]=[
37  49  54 ;
609 72  88 ;
939  1  21 ;
133 149 156 ]
(T,I,J)=size(time_distances)
```

Notice: This time, firstly a 3-dimensional matrix of zero values is created. Then the two two-dimensional sub-matrices are filled out. Again we can access elements:

```
time_distance[2,2,3]
```

## 9.2 Including Julia data

The data section in a Julia program can become rather large and it may be convenient to store the data in a separate file. This can be done very simply by:

```
include("NetworkData.jl")
```



Notice that the data file has to have a Julia format and it is simply inserted into the Julia file at the place of the include statement. Notice that you may have to write the full path to the file.



# 10 Future reading

This book aims at teaching basic Mathematical Programming modelling, both for LP models and MIP models. However, there are a lot of important subjects regarding Mathematical Programming and the broader field of Operations Research that have been omitted. In this section, we will try to give pointers to material for further studies.

The most used introductory textbook globally on Operations Research is "Introduction to Operations Research" Hillier and Lieberman (2020). This book introduces a lot of topics in Operations Research, including LP and MIP. This book offers a good starting place for anyone who would like a broader introduction to Operations Research.

For a good and more mathematical introduction to the mathematics behind solving MIP, we recommend "Integer Programming" Wolsey (2020).

In our opinion, a good book introducing Julia as a programming language, is still missing. The best material is available online at [julialang.org](https://julialang.org), which is also kept up to date.

## 10.1 Classic OR problems

Most real-world optimization problems are both unique and often very similar. The similarity arises by including parts of a list of classical, well-known Operations Research problems. A number of the exercises in this book are actually formulations of these classic problems, see the list below:

- **The Project Scheduling Problem:** Project Scheduling 4.6
- **The Knapsack Problem:** Startup Fund 6.2.
- **The Set Packing/Set Partitioning Problem:** Stamps 6.4
- **The Bin Packing Problem:** Scrap Removal 6.10
- **The Graph Coloring Problem:** Food Festival 6.16

- **The Lot Sizing Problem:** Micro Brewery 2 6.5
- **The Chinese Postman Problem:** Tennis 6.12
- **The Travelling Salesman Problem:** Groceries 6.11
- **The Vehicle Routing Problem:** Groceries 6.11

## 10.2 Optimization hardness

As mentioned in Chapter 5, the required solution times for MIP models above a certain problem size increases dramatically. The theoretical explanation for this increased solution time is based on the computational complexity theory  $P \neq NP$  Garey and Johnson (1979). While computational complexity theory deals with **decision problems** and MIP models handles optimization problems, many of the MIP models are generalizations of so-called NP-complete decision problems. Therefore, the exponential increase in solution time of MIP models is unavoidable. The good news is, however, that tremendous improvements have been achieved in the last decades Bixby (2012), so that even though a problem may be theoretically hard to solve, MIP models of the relevant size can be solved. Unfortunately, it is impossible to predict how hard it is for a MIP model solver to solve a problem because the complexity of the solvers has grown significant and so has their behaviour on different MIP models, despite the improved average performance Lodi (2013). The easiest way to find out whether a MIP model can be solved is simply to implement it.

## Author biographies



**Richard Lusby** is an Associate Professor at the Technical University of Denmark and teaches courses in Operations Research, Mathematical Modelling, and Decomposition Methods. His main research interests include Integer Programming, Decomposition Methods, and Metaheuristics, and their application to public transportation and manpower planning problems.



**Thomas Stidsen** is an Associate Professor at the Technical University of Denmark and teaches courses in Mathematical Modelling, Metaheuristics, and Decomposition Methods. His main research interests include optimization approaches for educational timetabling and manpower planning, and multi-objective optimization.



# Bibliography

- Aneja, Y. P. and Nair, K. P. (1979). Bicriteria transportation problem. *Management Science*, 25(1):73–78.
- Bixby, R. E. (2012). A brief history of linear and mixed-integer programming computation. *Documenta Mathematica*, 2012:107–121.
- Chalmet, L., Lemonidis, L., and Elzinga, D. (1986). An algorithm for the bi-criterion integer programming problem. *European Journal of Operational Research*, 25(2):292–300.
- Chankong, V. and Haimes, Y. Y. (2008). *Multiobjective decision making: theory and methodology*. Courier Dover Publications.
- Danzig, G. (1947). The dantzig simplex method for linear programming.
- Domínguez-Ríos, M. Á., Chicano, F., and Alba, E. (2021). Effective anytime algorithm for multiobjective combinatorial optimization problems. *Information Sciences*, 565:210–228.
- Garey, M. R. and Johnson, D. S. (1979). *Computers and intractability*, volume 174. freeman San Francisco.
- Hillier, F. S. and Lieberman, G. J. (2020). *Operations research concepts and cases*.
- Kirlik, G. and Sayın, S. (2014). A new algorithm for generating all nondominated solutions of multiobjective discrete optimization problems. *European Journal of Operational Research*, 232(3):479–488.
- Land, A. and Doig, A. (1960). An automatic method of solving discrete programming problems, *econometrica*, vol. 28, no. 3.
- Lodi, A. (2013). The heuristic (dark) side of mip solvers. In *Hybrid metaheuristics*, pages 273–284. Springer.
- Tamby, S. and Vanderpooten, D. (2021). Enumeration of the nondominated set of multiobjective discrete optimization problems. *INFORMS Journal on Computing*, 33(1):72–85.
- Wolsey, L. A. (2020). *Integer programming*. John Wiley & Sons.